

From Operating Systems to Agentic OS: Benchmarking, Perception, and Closed-Loop Control for GUI Agents

Wenqian Zhang, 225040483

Thesis Proposal

Academic Report

School of Data Science

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Modern computing systems have repeatedly advanced by introducing new abstraction layers that reduce the cognitive burden of using machines. Operating systems abstracted hardware resources, graphical user interfaces abstracted command-driven interaction, and contemporary multimodal agents suggest a further possibility: an agentic operating-system layer that accepts natural-language goals, observes real computer states, executes actions across applications, verifies outcomes, and recovers from errors. This report studies that trajectory through three connected works: OSWorld, CogAgent, and OSCAR. OSWorld establishes a realistic benchmark and executable environment for evaluating open-ended computer-use agents in real operating-system settings; CogAgent addresses a central perceptual bottleneck by improving high-resolution GUI understanding and navigation from screenshots; and OSCAR advances from perception toward closed-loop control by combining GUI-grounded observation, executable code-based action, state-aware reasoning, and task-driven re-planning.

Rather than treating these papers as isolated contributions, this report synthesizes them as successive steps toward general-purpose computer automation. The central argument is that robust agentic OS control requires three mutually dependent capabilities: realistic execution-based evaluation, fine-grained GUI grounding, and adaptive control under dynamic feedback. OSWorld reveals that current LLM/VLM agents remain far below human performance in real computer environments, especially because of GUI grounding errors, operational knowledge gaps, noise sensitivity, and execution drift. CogAgent shows that specialized high-resolution visual-language modeling can substantially improve GUI percep-

tion and action prediction. OSCAR then demonstrates that perception alone is insufficient: practical OS automation also requires a closed-loop runtime that can observe, plan, execute, verify, and re-plan when the environment changes. Together, these works clarify both the promise and the remaining limitations of LLM-based computer-use agents, pointing toward future systems in which agent runtimes, safety boundaries, and operating-system interfaces become central research objects.

Contents

Abstract	ii
1 Introduction	2
1.1 Research Questions	4
1.2 Three Existing Problems	6
1.3 Report Contributions	7
1.4 Methodological Posture and Scope	7
2 Background and Motivation	9
2.1 The Operating System as a Human Abstraction	9
2.2 From Language Agents to Computer-Use Agents	10
2.3 Related Work: From Web Agents to OS Agents	11
2.4 Why Benchmarks Come First	12
2.5 Why GUI Perception Is a Distinct Problem	13
2.6 Why Closed-Loop Control Is Necessary	14
3 Problem Definition: Agentic OS Control	15
3.1 Task Setting	15
3.2 Observation Space	16
3.3 Action Space	17
3.4 Evaluation Space	17
3.5 Core Challenges	18
4 Overview of the Three Works	20
4.1 OSWorld: A Benchmark for Real Computer Tasks	20
4.2 CogAgent: A Visual Language Model for GUI Agents	21
4.3 OSCAR: State-Aware Reasoning and Re-Planning	22
4.4 A Unified Storyline	23
5 Detailed Methodology	26
5.1 OSWorld Methodology	26
5.2 CogAgent Methodology	27

5.3	OSCAR Methodology	28
5.4	Methodological Relationship	29
6	Cross-Paper Comparison and Synthesis	35
6.1	Benchmark, Perception, and Control	35
6.2	Comparison by Environment Realism	36
6.3	Comparison by Observation and Grounding	37
6.4	Comparison by Action Representation	37
6.5	Comparison by Feedback and Re-Planning	39
6.6	Comparison by Limitations	39
7	Experimental Results and Analysis	41
7.1	Why More Tables Are Needed	41
7.2	OSWorld: Baseline Gap on Real Computer Tasks	41
7.3	CogAgent: GUI Perception and Navigation Evidence	42
7.4	OSCAR: Dynamic Control Results	44
7.5	Cross-Paper Experimental Lessons	46
8	Discussion and Future Directions	50
8.1	Strengths of the Three-Work Trajectory	50
8.2	Weaknesses and Remaining Limitations	51
8.3	Validity of the Cross-Paper Comparison	52
8.4	Benchmark and Evaluation Risks	53
8.5	Perception and Action Risks	53
8.6	Security and Runtime Boundaries	54
8.7	Case Study: OpenClaw and ClawJacked	54
8.8	Future Direction: Better Grounding	55
8.9	Future Direction: Memory and Operational Knowledge	56
8.10	Future Direction: Verification and Recovery	56
8.11	Future Direction: Human-Agent Collaboration	57
9	Conclusion	58
	References	60

List of Figures

1.1	OSWorld frames computer-use evaluation as open-ended interaction with real applications, initial states, and execution-based evaluators. The figure is reused from the OSWorld source material to anchor the report’s benchmark-first storyline.	5
4.1	OSWorld provides environment setup and execution-based evaluation for open-ended computer-use tasks. This figure supports the report’s claim that benchmark realism must include both state initialization and outcome checking.	21
4.2	CogAgent demonstrates visual-agent behavior over GUI-centric tasks. The figure is useful in this report because it makes the move from language-only agents to visually grounded agents concrete.	24
4.3	OSCAR contrasts static and dynamic OS environments. The same goal may admit multiple valid action trajectories, so final-state verification is more appropriate than rigid reference-action matching.	25
5.1	CogAgent architecture. The model combines a general visual-language backbone with a high-resolution branch so that global layout and fine-grained GUI details can both contribute to reasoning.	30
5.2	Examples of GUI grounding data used by CogAgent. This figure illustrates why GUI pretraining is different from ordinary image-caption pretraining: the model must connect instructions, text, layout, and actionable visual elements.	31
5.3	CogAgent’s FLOPs comparison illustrates the trade-off between high-resolution GUI perception and computational efficiency. The report uses this figure to explain why GUI perception requires architectural design rather than simple input scaling.	32
5.4	OSCAR’s state machine formalizes OS control as repeated observation, planning, execution, verification, and re-planning. This figure is central to the report’s closed-loop control analysis.	32

5.5	OSCAR’s GUI-grounded observation combines raw screenshots with semantic and visual grounding inputs. This multimodal observation design helps the agent identify actionable elements.	33
5.6	OSCAR’s task-driven re-planning refines the current task list and action choice using observation, memory, and feedback. It is designed to avoid restarting the entire workflow after every failure.	33
5.7	A qualitative OSCAR example from the source paper. The example shows the kind of multi-step GUI workflow for which state-aware observation, code-centric action, and re-planning are useful.	34
6.1	A synthesized pipeline for agentic OS control derived from the slide deck and the three core papers. The report’s thesis is that an agentic OS must be measurable, perceptive, controllable, and secure.	37
7.1	OSWorld analysis of agent step counts. The figure helps explain why long-horizon tasks are difficult: more steps create more opportunities for grounding errors, wrong operations, and state drift.	43
7.2	OSWorld robustness analysis under UI perturbations. The figure supports the report’s claim that real OS agents must handle noise, unexpected windows, and visual-state variation rather than assuming a clean static screen.	47
7.3	CogAgent failure cases from the source material. These examples show that even a GUI-specialized VLM can still struggle with precise interaction, ambiguity, or visual reasoning under complex interface states.	48
7.4	OSCAR ablation decomposition. The figure visualizes the contribution of GUI-grounded observation and task-driven re-planning, supporting the claim that both perception and control-loop design are necessary.	48
7.5	OSCAR planning-efficiency analysis for successful cases. The figure illustrates why fine-grained re-planning can be more efficient than repeatedly restarting an entire workflow.	49
8.1	A successful OSWorld qualitative example. The case demonstrates that current agents can sometimes solve complex real-computer tasks when perception, planning, and operational knowledge align.	51
8.2	OSWorld qualitative failure examples. These cases illustrate grounding failure, missing operational knowledge, and action-state mismatch, which motivate the need for both GUI-specialized perception and closed-loop recovery. . . .	52

List of Tables

2.1	Positioning of related work around the agentic OS problem. The three core papers occupy complementary roles: OSWorld defines the evaluation setting, CogAgent improves GUI perception, and OSCAR designs a closed-loop control runtime.	13
6.1	Functional positioning of the three core papers in the agentic OS trajectory. .	36
6.2	Comparison of observation, action, feedback, and evaluation assumptions. .	38
6.3	Limitations and open problems revealed by the cross-paper synthesis.	40
7.1	Representative OSWorld baseline results. Success rates are grouped by task category. The table highlights the large gap between current LLM/VLM agents and human users in real-computer environments.	42
7.2	CogAgent results on Mind2Web. The comparison shows that screenshot-based GUI perception can outperform HTML-based LLM methods in this setting.	43
7.3	CogAgent results on Android in the Wild. The table shows that CogAgent improves over textual-description baselines and Auto-UI on the overall AITW score.	44
7.4	CogAgent ablation on pre-training data. Adding GUI and grounding data produces a large Mind2Web improvement, supporting the claim that GUI-specific data matters.	44
7.5	OSCAR results on GAIA. The table reports exact-match scores across increasing workflow complexity.	45
7.6	OSCAR results on OSWorld. All baselines use SoM visual prompts and GPT-4o in the OSCAR paper’s comparison.	45
7.7	OSCAR results on AndroidWorld. The table shows that OSCAR improves average performance over AppAgent, Mobile Agent, and M3A in the reported setting.	45

Chapter 1

Introduction

Computing history can be read as a sequence of abstraction layers designed to make increasingly complex machines usable by humans. Early resident monitors and time-sharing systems reduced the difficulty of managing scarce hardware resources. General-purpose operating systems such as OS/360 and UNIX provided systematic abstractions over processes, files, devices, and users. Later, graphical user interfaces reduced the need to remember textual commands by exposing software functionality through windows, icons, menus, and pointers. Each transition preserved the underlying complexity of computation while changing how humans expressed intent. The slide deck that guides this report frames the current moment in the same historical pattern: as applications, workflows, credentials, files, and interfaces proliferate, natural-language agents may become another abstraction layer above the conventional operating system.

The motivation for such an “agentic OS layer” is straightforward. Modern computer work is rarely confined to a single application or a single API. A realistic task may require searching the web, editing documents, moving files, using spreadsheets, operating professional software, switching windows, interpreting visual feedback, and correcting errors along the way. Human users perform these tasks by combining semantic intent with situated perception and low-level control. They know not only what should be done, but also where to click, when the screen state has changed, whether an operation succeeded, and

how to recover when a window appears unexpectedly. Large language models have made high-level instruction following and planning far more capable, but computer-use agents still face the difficult problem of grounding those plans in real graphical and command-line environments.

This report studies three works that together define a coherent research path toward such agents. The first is OSWorld, a benchmark and scalable real-computer environment for evaluating multimodal agents on open-ended tasks across operating systems and applications [22]. OSWorld is important not merely because it reports low baseline performance, but because it changes the evaluation target. Earlier GUI and web-navigation benchmarks often rely on static demonstrations or reference action traces, which can penalize alternative correct solutions and cannot fully measure whether a real task has been completed. OSWorld instead emphasizes executable environments, initial-state setup, and execution-based evaluators. In doing so, it exposes a large gap between human performance and existing LLM/VLM agents: humans solve a majority of tasks, while the best reported baseline in the paper reaches only a small fraction of that success rate. The benchmark therefore establishes the first stage of the story: before building an agentic OS layer, the field needs a realistic way to measure whether agents can actually use computers.

The second work, CogAgent, addresses one of the major failure modes revealed by this benchmark setting: GUI grounding [5]. Most applications are designed for human visual perception rather than machine-readable APIs. Buttons, menus, tabs, icons, toolbars, documents, and canvases are arranged spatially, and their meaning often depends on visual layout. Purely text-based agents struggle when HTML, accessibility trees, or APIs are unavailable, incomplete, or misleading. CogAgent therefore treats GUI interaction as a visual-language problem. It introduces an 18-billion-parameter visual language model specialized for GUI understanding and navigation, using both low-resolution and high-resolution visual encoders to preserve global layout and fine-grained interface details. Its high-resolution design targets a practical GUI fact emphasized in the slides: agents often know what to do, but fail because they cannot identify where to act. CogAgent’s

results on benchmarks such as Mind2Web and Android in the Wild show that improved screenshot-based perception can outperform methods that depend on extracted structured text.

The third work, OSCAR, moves from perception toward closed-loop control [20]. The slide deck presents this as the transition from “seeing the UI” to operating in a dynamic environment. In real computer use, a fixed action trace is fragile: another valid path may complete the same task, a dialog may appear, a file may be located elsewhere, or a previous click may fail. OSCAR responds to this problem by modeling OS control as a state-aware process. It observes the screen and GUI elements, plans actions, executes Python control code, verifies outcomes, and re-plans when feedback or errors indicate that the current trajectory is no longer appropriate. This design shifts the agent from static action prediction to adaptive execution. Its reported evaluations on GAIA, OSWorld, and AndroidWorld suggest that state-aware reasoning and task-driven re-planning can improve robustness across desktop, mobile, and workflow-oriented settings.

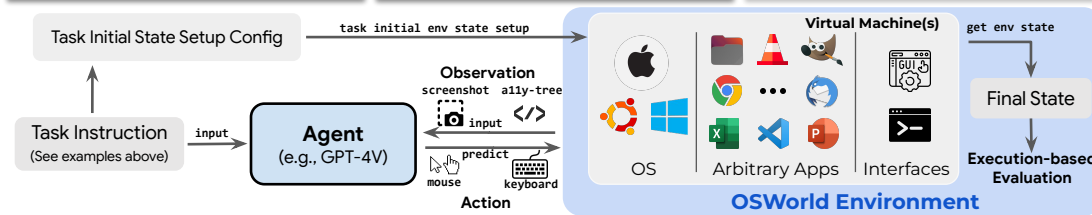
Taken together, these works suggest that an agentic OS layer cannot be reduced to a single model capability. OSWorld shows that the research community needs dynamic, execution-based benchmarks that reflect real computer complexity. CogAgent shows that agents need high-resolution multimodal perception to ground instructions in dense graphical interfaces. OSCAR shows that even strong perception must be embedded in an execution runtime that supports verification, error handling, and re-planning. The three papers therefore form a layered account of computer-use agents: benchmark the real environment, perceive the GUI accurately, and close the loop with adaptive control.

1.1 Research Questions

Following the slide deck’s central question, this report asks whether LLMs and VLMs can become a new “OS layer” for humans. This question is intentionally broader than whether a model can click a button or answer a screenshot question. An operating-system-like agent must mediate between human intent and executable system behavior. It must therefore

The figure consists of three side-by-side screenshots of a Windows desktop environment, illustrating the progression of a malware infection. Each screenshot shows a desktop with a taskbar at the bottom and a standard Windows interface.

- Left Screenshot:** A spreadsheet titled "Bookkeeping simple" is open. It contains a table with columns: Description, Category, Type, Amount, and Balance. The data includes entries for "Other Supplies", "Client Payment Received", "Interest Bill", "Freelance Services", "Rent Payment", and "Software Subscription". A mouse cursor is pointing at the "Software" category in the "Software Subscription" row.
- Middle Screenshot:** The desktop shows a "receipt" application window in the foreground and a "112" emergency services window in the background, suggesting a system lock or emergency call initiated by the malware.
- Right Screenshot:** The same "Bookkeeping simple" spreadsheet is open, but the data has been modified. The "Amount" and "Balance" columns now show different values, including negative amounts, indicating data manipulation or replacement by the malware.



be measurable, perceptive, controllable, and governable. The report organizes this broad question into four research questions:

- 5

These questions map directly onto the sequence OSWorld $\rightarrow$ CogAgent $\rightarrow$ OSCAR. In the terminology of the slide deck, the path is **measure** $\rightarrow$ **perceive** $\rightarrow$ **control**. OSWorld measures real computer-use ability; CogAgent improves the perceptual layer needed to identify where actions should occur; OSCAR turns perception into feedback-driven control. The final question extends this stack with a fourth requirement: **secure**. If the agent becomes an OS-like layer, then its runtime becomes part of the trusted computing base.

1.2 Three Existing Problems

The first existing problem is the **benchmark gap**. Many earlier datasets evaluate web navigation or GUI action prediction under static conditions. Such evaluations are useful but incomplete: they cannot always distinguish an agent that truly completed a task from one that merely followed a demonstration-like trajectory. OSWorld responds by constructing real operating-system tasks with executable evaluators, making final-state success the evaluation target [22].

The second existing problem is the **perception gap**. Modern GUIs are visually dense, text-rich, and spatially structured. Standard VLMs often lose small text and fine-grained UI elements when screenshots are downsampled. CogAgent addresses this by combining global layout perception with high-resolution detail perception and GUI-specific grounding data [5].

The third existing problem is the **execution gap**. Even if an agent can plan and perceive, real software environments change during execution. Dialogs appear, actions partially succeed, files move, and mistakes accumulate. OSCAR addresses this by treating OS control as a state-aware loop: observe, plan, execute, verify, and re-plan [20]. This execution gap explains why perception-only systems remain insufficient for reliable computer automation.

1.3 Report Contributions

This report makes three contributions as a synthetic academic review. First, it reconstructs the three papers as a single research trajectory rather than three separate summaries: OSWorld supplies the benchmark layer, CogAgent supplies the perception layer, and OSCAR supplies the control layer. Second, it develops a comparative framework for analyzing computer-use agents across environment realism, observation representation, action representation, feedback, evaluation, and security. Third, it identifies open problems that remain after the three works, especially benchmark scalability, GUI grounding uncertainty, runtime safety, reproducibility, and human-agent collaboration. The resulting thesis is concise: an agentic OS must be measurable, perceptive, controllable, and secure.

1.4 Methodological Posture and Scope

This report is a critical synthesis of provided papers, arXiv source materials, and the accompanying slide deck. It does not claim to reproduce the original experiments or introduce new benchmark results. Quantitative claims are attributed to the original papers and should be interpreted within their reported experimental settings. Cross-paper comparisons are used to clarify research structure, not to assert strict apples-to-apples superiority, because OSWorld, CogAgent, and OSCAR differ in goals, benchmarks, action spaces, and runtime assumptions. When the report uses figures or tables from the source papers, they are included to support explanation of the argument rather than to substitute for the papers' full experimental context.

The rest of this report develops this synthesis in detail. Chapter 2 introduces the background and motivation for agentic operating-system abstractions. Chapter 3 defines the problem of computer-use agent control in dynamic GUI environments. Chapter 4 gives a high-level overview of OSWorld, CogAgent, and OSCAR. Chapter 5 examines their technical designs in greater detail. Chapter 6 compares the three works across evaluation realism, perception, action representation, feedback, and robustness. Chapter 7 analyzes the

experimental results and what they imply about current bottlenecks. Chapter 8 discusses limitations, safety concerns, and future directions. Chapter 9 concludes the report.

Chapter 2

Background and Motivation

2.1 The Operating System as a Human Abstraction

The conventional operating system has two identities. Technically, it is a resource manager that multiplexes processors, memory, storage, devices, and network access. Socially, it is a human-facing abstraction that lets users and programmers reason about a machine without thinking directly in terms of circuits, device registers, or scheduling details. The second identity is crucial for this report. The history summarized in the slide deck, from resident monitors to time sharing, OS/360, UNIX, and graphical user interfaces, can be understood as a gradual relocation of complexity. As computers gained power and generality, users needed increasingly powerful abstractions to make that complexity manageable.

The GUI era made this relocation visible. Instead of requiring users to memorize command syntax, the system exposed actions as visual affordances. A document could be opened by double-clicking, a file could be moved by dragging, and a printer could be selected from a dialog. This abstraction made computing broadly accessible, but it also introduced a new dependency: the user must perceive and manipulate a visual interface. Modern digital labor is therefore not only cognitive but also operational. A user must understand the goal, know which application can satisfy it, locate the relevant interface elements, sequence actions correctly, and recover from mistakes.

The proposed idea of an agentic OS layer extends this logic. If an operating system abstracts hardware and a GUI abstracts commands, an agent may abstract workflows. Such a layer would not replace the kernel or the desktop environment. Rather, it would sit above them as an intent-to-action mediator. The user would express a goal in natural language; the agent would inspect the current system state; it would plan and execute interactions across applications; it would verify whether the goal had been achieved; and it would respect boundaries around credentials, files, destructive actions, and untrusted instructions. In this sense, an agentic OS is less a new kernel than a new control plane for everyday computing.

2.2 From Language Agents to Computer-Use Agents

Large language models made agentic workflows plausible because they can parse instructions, decompose tasks, write code, and reason over tool outputs. However, a language-only agent is poorly matched to the way most software is actually used. Many desktop and mobile applications do not expose clean APIs for every user-facing function. Even when APIs exist, users often describe tasks in terms of visible interface states rather than internal objects. A command such as “center-align the title in this document” requires document semantics, visual grounding, and operational knowledge about the word processor. A command such as “convert this presentation to a video and play it” may require multiple applications, file dialogs, export settings, and final verification.

GUI agents attempt to bridge this mismatch by combining language reasoning with visual observation and low-level actions such as clicking, typing, scrolling, dragging, and pressing keyboard shortcuts. The central challenge is that GUI interaction is both semantic and spatial. The agent must understand that a button labelled “Print” is relevant, but it must also select the correct target coordinate or accessibility node. It must know that a window is active, that a menu is open, that a modal dialog blocks the underlying application, or that a file has been saved successfully. These details are routine for human users but brittle for models trained primarily on text or natural images.

The three works studied in this report can be arranged around the core question: what

must be true before an LLM or VLM can function as a general computer assistant? OSWorld answers that the field first needs an executable benchmark that reflects real computer use. CogAgent answers that agents need visual-language models adapted to dense, high-resolution GUI perception. OSCAR answers that an agent must close the loop with feedback, verification, and re-planning rather than predicting a single static action trajectory.

2.3 Related Work: From Web Agents to OS Agents

The broader agent literature helps explain why the three selected papers form a meaningful path. Early and contemporary language-agent systems show that LLMs can decompose goals, call tools, and reason over intermediate observations. Frameworks and prompting patterns such as ReAct-style tool use, reflection, and multi-step planning made it natural to view an LLM as more than a text generator. General assistant benchmarks such as GAIA further emphasize that useful agents must combine reasoning, tool use, retrieval, and verification in multi-step workflows [9]. However, these settings do not by themselves solve the problem of direct GUI operation.

Web-agent benchmarks and datasets narrow the problem by grounding agents in browser interaction. Mind2Web studies instruction-following over real websites and highlights the difficulty of selecting correct web elements from complex pages [3]. Other work on web navigation and interactive agents similarly shows that text-only reasoning is fragile when the environment is visual, stateful, and only partially structured. These tasks are important predecessors of computer-use agents because many desktop workflows now pass through browsers, but web pages are still only one part of an operating-system environment.

Mobile and GUI-agent research broadens the scope beyond web pages. Android in the Wild and AndroidWorld evaluate interaction with mobile applications, where screenshots, touch actions, and application state are central [13, 14]. AppAgent and Mobile-Agent explore agent designs that operate mobile apps through visual observation and action histories [18, 24]. These works make the same conceptual move as the slide deck: the agent must use software in the form in which users encounter it, not only through idealized APIs.

Multimodal grounding research then supplies the perception foundation. Set-of-Mark style annotation makes visual references more explicit by overlaying candidate marks on the screen [23]. CogVLM provides the visual-language foundation from which CogAgent is developed [19]. CogAgent extends this direction toward GUI-specific high-resolution perception, which is why it fits naturally between OSWorld and OSCAR in the report’s storyline.

Finally, OS-level agent systems investigate how these capabilities become an operating-system control plane. UFO and related work on Windows or desktop copilot agents examine how an agent can interact with applications and system state rather than isolated webpages [21, 26]. OSCAR belongs to this line but places particular emphasis on state-machine control, executable action generation, and dynamic re-planning. The result is a shift from model benchmarking to runtime design.

2.4 Why Benchmarks Come First

The slide deck places OSWorld before CogAgent and OSCAR for a methodological reason. Without a realistic benchmark, progress on GUI agents can be misleading. Static datasets are useful for controlled comparisons, but they often define success as matching a reference action or selecting a gold element. This is reasonable for measuring a narrow prediction problem, yet it differs from the user’s real objective. Many computer tasks allow multiple correct paths. A user may open a file through a file explorer, a recent-file menu, a command line, or an application-specific search box. A static benchmark may treat only one path as correct, while a real operating system should judge the final outcome.

Execution-based evaluation also reveals failures that are invisible in text-based evaluation. An agent may produce a convincing plan but click the wrong coordinate. It may open the correct application but leave the file unsaved. It may perform an operation in the wrong window. It may recover from one error but create another. These are not small implementation details; they are the substance of computer use. OSWorld therefore reframes the evaluation problem around real state transitions and final task completion. Its custom

Table 2.1: Positioning of related work around the agentic OS problem. The three core papers occupy complementary roles: OSWorld defines the evaluation setting, CogAgent improves GUI perception, and OSCAR designs a closed-loop control runtime.

Research line	Representative works	Main contribution	Limitation for agentic OS
General language agents	GAIA and tool-use agent benchmarks [9]	Measure multi-step reasoning, retrieval, and tool use	Often abstract away direct GUI manipulation and OS state
Web navigation	Mind2Web and related web-agent datasets [3]	Ground instructions in complex webpages	Browser-centric and often less general than full desktop workflows
Mobile GUI agents	AITW, AndroidWorld, AppAgent, Mobile-Agent [13, 14, 18, 24]	Evaluate app interaction through screenshots and touch actions	Mobile interfaces do not cover the full desktop OS and cross-application file workflows
GUI grounding	Set-of-Mark, CogVLM, CogAgent [5, 19, 23]	Improve perception of visual elements, text, and layout	Perception does not by itself guarantee reliable task completion
OS-level agents	OSWorld, UFO, OSCAR [20, 22, 26]	Evaluate and control real operating-system environments	Reliability, safety, and reproducibility remain open

evaluators check files, applications, UI states, and workflow outcomes rather than measuring linguistic similarity.

2.5 Why GUI Perception Is a Distinct Problem

The second motivation is perceptual. GUIs are text-rich and detail-heavy. Important information can be contained in small labels, icons, tabs, table cells, cursor positions, disabled buttons, color states, and spatial grouping. Standard VLM resolutions are often too low to read tiny interface text or distinguish small controls. Increasing image resolution naively increases computation, because high-resolution images expand the visual token sequence. CogAgent’s architectural emphasis on combining global layout perception with high-resolution detail perception directly addresses this tension.

GUI perception also differs from natural-image perception. A natural-image model may learn objects, textures, and scenes, but GUI interaction requires recognizing affordances and operational roles. A rectangle with text may be a button, a tab, a selected cell, a text field, a notification, or a disabled control. The meaning depends on layout and application conventions. Thus, GUI agents benefit not only from high-resolution input, but also from GUI-specific training data that teaches the model to align user intent with screen elements.

2.6 Why Closed-Loop Control Is Necessary

The third motivation is control. A perceptive agent may still fail if it executes open-loop action sequences. Real OS environments are dynamic. Windows move, applications load slowly, dialogs interrupt workflows, files may be absent, and previous actions may partially succeed. Static action prediction assumes that the next step can be chosen from a known state distribution, but real computer use requires adaptation.

OSCAR’s state-machine formulation addresses this issue by making feedback explicit. The agent does not simply generate an entire plan and hope that it works. It observes, plans, executes, verifies, and re-plans. It can incorporate interpreter errors, verification failures, and updated screenshots into the next decision. This structure resembles human troubleshooting: when an action fails, the user does not restart all reasoning from scratch, but revises the relevant subtask in light of the new state. The report therefore treats closed-loop control as the third pillar of agentic OS research.

Chapter 3

Problem Definition: Agentic OS Control

3.1 Task Setting

This report uses the term *agentic OS control* to describe the problem of enabling an AI agent to complete high-level user tasks by interacting with a real or realistically simulated operating-system environment. The agent receives a user instruction, observes the current computer state, chooses actions, executes them through available input channels, and determines whether the requested goal has been satisfied. The setting is broader than web navigation and narrower than general artificial intelligence. It is broader because desktop and mobile computing involve many applications, files, settings, and interfaces. It is narrower because the agent is grounded in a specific operational substrate: screens, windows, accessibility trees, files, applications, keyboard events, mouse events, and sometimes command-line interfaces.

Formally, an agentic OS task can be described as a tuple consisting of a user goal, an initial system state, an observation function, an action space, a transition process, and an evaluation function. The user goal is usually a natural-language instruction. The initial state specifies the environment before the task begins, including application state, file state,

window state, installed software, and sometimes account or credential state. The observation function exposes the current environment to the agent through screenshots, accessibility information, OCR, structured UI trees, terminal output, or other signals. The action space defines how the agent can affect the system: moving and clicking the mouse, typing text, pressing shortcuts, scrolling, dragging, invoking APIs, or executing code. The transition process is the operating system and applications responding to those actions. The evaluation function determines whether the task has been completed.

This formulation highlights why the problem is hard. The transition process is not fully under the agent’s control and may be nondeterministic. The observation may be incomplete or noisy. The action space is low-level relative to the user’s goal. The evaluation may require inspecting files, UI states, or application-specific outputs. Most importantly, there may be many valid trajectories from the same initial state to the same goal state. A robust agent must therefore reason over goals and outcomes, not merely imitate a single demonstration.

3.2 Observation Space

The observation space is the agent’s window into the computer. At minimum, a GUI agent can receive a screenshot. Screenshots preserve what a human sees, including visual layout, icons, images, selected states, and non-textual affordances. However, screenshots also present difficulties: small text may be unreadable, coordinates must be inferred, and the model must identify actionable elements from pixels. Additional representations such as OCR text, accessibility trees, and Set-of-Mark prompts can make the screen easier to ground, but they are not universally reliable. Accessibility information may omit custom canvas elements; OCR may misread small or stylized text; and structured trees may expose too many irrelevant nodes.

The three papers place different emphasis on observation. OSWorld evaluates multiple agent settings, including screenshot-based and accessibility-tree-based observations, and shows that neither alone solves the problem. CogAgent focuses on the screenshot route and improves visual perception with high-resolution modeling. OSCAR combines screenshot

observation with semantic grounding and Set-of-Mark visual grounding, using native system APIs when available. Together, these choices suggest that agentic OS control benefits from multi-representation observation: screenshots preserve human-visible context, while structured signals make target selection more precise.

3.3 Action Space

The action space determines what kind of control the agent has over the system. A simple GUI-navigation benchmark may ask the model to predict a predefined action such as click, type, scroll, or stop. A real OS agent, however, may need richer control. It may click a target by coordinate or element identifier, press keyboard shortcuts, write text, drag objects, execute shell commands, or run Python scripts that call OS automation libraries. The action representation must balance expressiveness and safety. If actions are too restricted, the agent cannot handle real workflows. If actions are too unconstrained, the agent may produce invalid or unsafe behavior.

OSCAR’s code-centric control illustrates the expressive side of this design. By translating user goals into executable Python control code, the agent can use precise operations and reuse OS automation capabilities. The risk is that executable actions must be verified, sandboxed, and constrained. The action channel becomes part of the security boundary. For a future agentic OS layer, action design is not only a modeling problem but also a systems problem involving permission, identity, rollback, and auditability.

3.4 Evaluation Space

Evaluation is one of the defining differences between static GUI benchmarks and OS-level agent benchmarks. In static datasets, the agent may be judged by whether it selects the same element or action as a reference trajectory. This is useful for measuring action prediction, but it does not necessarily measure task completion. In dynamic environments, the agent can be judged by the final state. If the requested file exists, the document has the correct

formatting, the application shows the correct output, or the workflow produces the intended artifact, then the task can be considered successful even if the path differs from a human demonstration.

OSWorld’s contribution is especially important here. It uses custom execution-based evaluators for real computer tasks. This shifts evaluation from imitation to accomplishment. OSCAR inherits this dynamic perspective and reports performance on OSWorld and AndroidWorld, where the agent must operate with real-time feedback. The report therefore treats execution-based evaluation as a necessary foundation for agentic OS research. Without it, the field risks optimizing agents that look competent on static traces but fail when deployed in live environments.

3.5 Core Challenges

The inspected papers and slides converge on five core challenges.

First, **GUI grounding** remains difficult. Agents must map language such as “the search box,” “the title,” or “the print button” onto a specific visual element or coordinate. Dense interfaces, small text, overlapping windows, and custom widgets make this hard.

Second, **operational knowledge** is uneven. A model may understand a goal but not know the procedure inside a professional application. OSWorld reports failures in software such as GIMP, LibreOffice Calc, and VS Code where the agent lacks practical knowledge of interface conventions.

Third, **trajectory length** amplifies errors. A single wrong click can move the environment into an unexpected state. Multi-application workflows are especially fragile because they require context transfer across software boundaries.

Fourth, **feedback integration** is essential. A static plan cannot anticipate every window, delay, or failure. The agent needs mechanisms to observe the result of an action, verify progress, and revise only the relevant part of its plan.

Fifth, **safety and security** become central once agents act in real operating systems. Agents may operate with user credentials, mutate files, run code, access private data, or

execute instructions embedded in untrusted content. The runtime, not only the model, becomes a boundary that must be designed and governed.

These challenges motivate the structure of the rest of the report. OSWorld makes the challenges measurable, CogAgent attacks the perception bottleneck, and OSCAR introduces a runtime structure for closed-loop control.

Chapter 4

Overview of the Three Works

4.1 OSWorld: A Benchmark for Real Computer Tasks

OSWorld is the benchmark-first component of the storyline. It introduces a scalable real-computer environment for multimodal agents, supporting task setup, execution-based evaluation, and interactive learning across operating systems [22]. The benchmark contains 369 real-world computer tasks involving web and desktop applications, operating-system file I/O, and workflows spanning multiple applications. The tasks are drawn from realistic computer use rather than isolated toy interfaces. Each task includes an initial-state setup and a custom evaluator, enabling repeatable assessment of whether the agent actually completed the requested work.

The core insight of OSWorld is that current agents remain far from human-level computer use. The paper reports that humans achieve 72.36% overall success, while the best baseline reaches 12.24% overall success in the reported setting. Workflow tasks that require multiple applications are even more difficult. These numbers are important because they shift attention away from isolated demonstrations of impressive behavior and toward systematic measurement. They show that general computer assistance is not solved by simply connecting a strong model to a screen.

OSWorld also identifies failure modes. Agents struggle with GUI grounding, visual

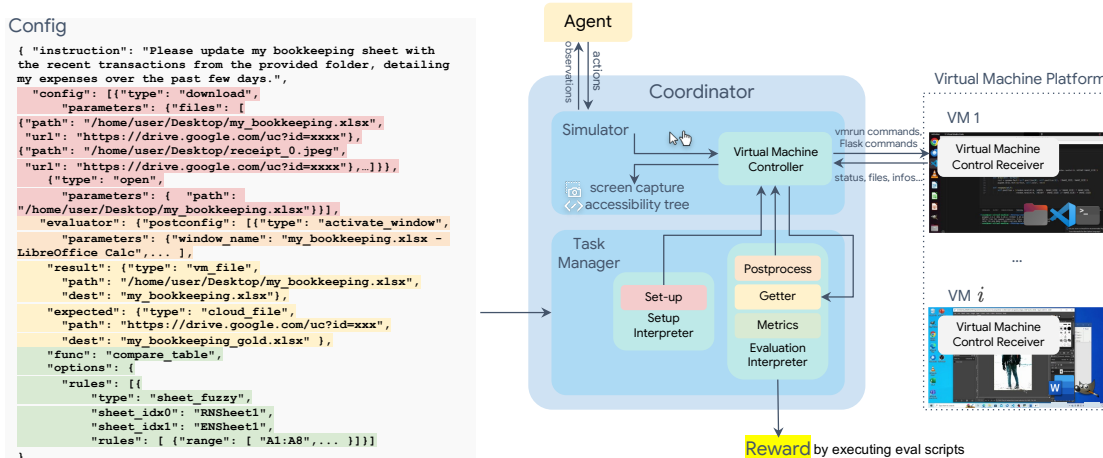


Figure 4.1: OSWorld provides environment setup and execution-based evaluation for open-ended computer-use tasks. This figure supports the report’s claim that benchmark realism must include both state initialization and outcome checking.

clutter, unexpected windows, repetitive actions, and insufficient operational knowledge. They may propose good high-level plans but fail to execute them step by step. This distinction between planning and execution becomes a central theme of the report. A model can know what should be done while failing to perform the exact interaction required by the current UI state.

4.2 CogAgent: A Visual Language Model for GUI Agents

CogAgent is the perception-centered component of the storyline. It introduces an 18-billion-parameter visual language model designed for GUI understanding and navigation [5]. The model is motivated by a limitation of language-only agents: most real software is mediated by GUIs, and GUI information cannot be fully represented by text. Icons, layout, spatial relations, visual grouping, and tiny interface labels are all relevant to action selection.

CogAgent’s architectural contribution is its use of both low-resolution and high-resolution visual encoders. The low-resolution pathway captures global layout, while the high-resolution cross-module preserves fine-grained details such as small text and icons. The model supports 1120 by 1120 inputs, which is important because standard VLM input sizes

may lose key GUI information. The design also addresses compute: naively processing high-resolution screenshots would be expensive, so CogAgent separates visual pathways to maintain detail without simply scaling all computation.

The paper reports state-of-the-art performance on multiple text-rich and general VQA benchmarks, and strong results on GUI navigation benchmarks such as Mind2Web and AITW. For this report, the most important finding is not only that CogAgent improves benchmark scores, but that GUI-specialized data and high-resolution perception matter. The ablation evidence described in the slides indicates that adding GUI and grounding data improves navigation performance beyond captioning and OCR alone. This supports the broader claim that an agentic OS layer needs models trained for interface semantics, not only generic visual understanding.

4.3 OSCAR: State-Aware Reasoning and Re-Planning

OSCAR is the control-centered component of the storyline. It proposes a generalist agent for operating-system control through state-aware reasoning and re-planning [20]. Unlike a perception-only model or a static action predictor, OSCAR treats computer use as a dynamic loop. It receives a user request, observes the environment, plans a task structure, executes actions through code-centric control, verifies progress, and re-plans when feedback indicates failure.

OSCAR identifies three key challenges for generalist OS agents: unified control interfaces, GUI grounding, and exploration-based simulation with re-planning. Unified control matters because an agent must operate standard input mechanisms across applications. GUI grounding matters because the agent must connect instructions to on-screen elements. Re-planning matters because real computer environments change and errors occur. The system addresses these challenges through GUI-grounded observation, a state machine, context memory, task-driven re-planning, and executable Python actions.

Empirically, OSCAR reports improved performance on GAIA, OSWorld, and Android-World. The paper states that OSCAR achieves a 28.7% average success rate on GAIA, 24.5%

on OSWorld, and 61.6% on AndroidWorld. These results do not close the gap to reliable human-like automation, but they demonstrate that dynamic feedback and re-planning can improve over simpler baselines. OSCAR therefore completes the three-step narrative: benchmark realism reveals the gap, GUI perception narrows one bottleneck, and state-aware control addresses the fragility of execution.

4.4 A Unified Storyline

The slide deck summarizes the three works as “Benchmark, Perception, Control loops.” This report adopts that structure. OSWorld asks whether agents can be measured in real computer environments. CogAgent asks how agents can see and ground GUI elements accurately. OSCAR asks how agents can act, verify, and recover. The three works are not substitutes for one another; they are complementary layers. A system like OSCAR benefits from benchmark environments such as OSWorld and from perceptual advances such as CogAgent. Conversely, benchmark results and control-loop failures reveal where perception models must improve.

This unified storyline matters because it avoids a common misunderstanding: agentic OS research is not only about building a better model. It is about constructing a full stack that includes tasks, environments, observations, action representations, evaluators, memory, planning, verification, and safety boundaries. The three papers collectively move the field from impressive but narrow demonstrations toward a more systematic account of computer-use agents.



Figure 4.2: CogAgent demonstrates visual-agent behavior over GUI-centric tasks. The figure is useful in this report because it makes the move from language-only agents to visually grounded agents concrete.

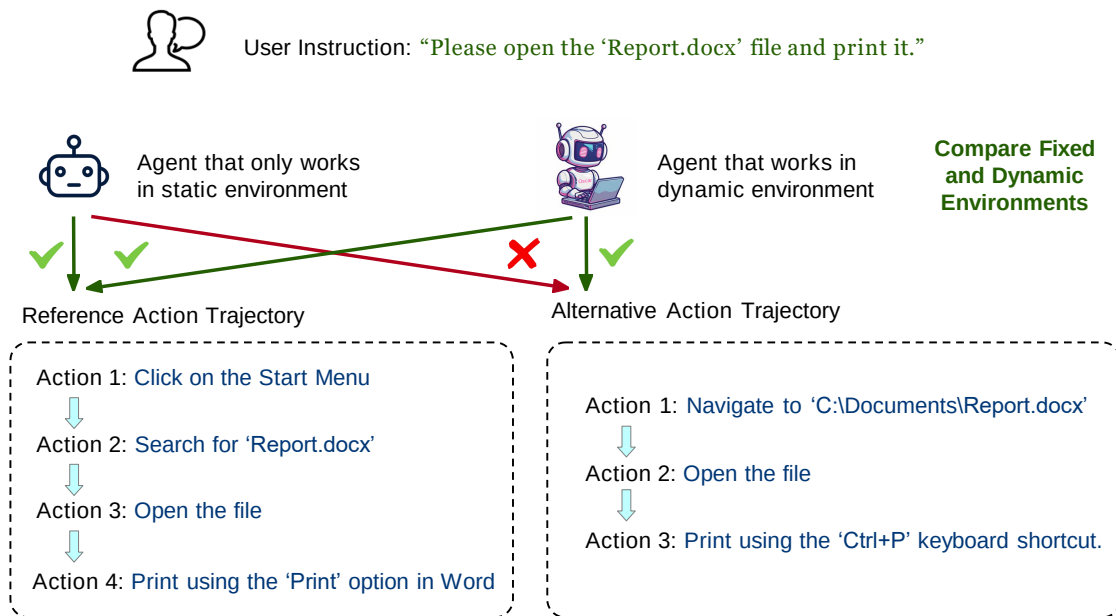


Figure 4.3: OSCAR contrasts static and dynamic OS environments. The same goal may admit multiple valid action trajectories, so final-state verification is more appropriate than rigid reference-action matching.

Chapter 5

Detailed Methodology

5.1 OSWorld Methodology

OSWorld’s methodology can be understood as an environment-and-evaluation design rather than a single model architecture. Its goal is to make real computer-use tasks reproducible, diverse, and automatically measurable. To do so, it builds around three components: an executable computer environment, task initialization, and execution-based evaluation.

The executable environment gives agents access to real applications rather than simplified web forms or synthetic interfaces. The paper emphasizes Ubuntu, Windows, and macOS support, though the benchmark’s reported tasks are centered on realistic desktop use. The environment exposes observations such as screenshots and accessibility information, and it permits actions through mouse and keyboard control. This is crucial because operating-system tasks involve multiple interface modalities. A user may need to open a file manager, edit a document, run a browser, use a terminal, or manipulate an application-specific toolbar.

Task initialization makes evaluation repeatable. Real computer-use tasks depend on the starting state. A file may already exist, an application may be open, a browser profile may contain cookies, or a window may be arranged in a particular way. OSWorld therefore annotates tasks with setup configurations that recreate a work-in-progress situation. This is more realistic than assuming every task begins from a clean desktop, and it is more rigorous

than evaluating agents in uncontrolled local states.

Execution-based evaluation is the benchmark’s most important methodological contribution. Each task is paired with a custom evaluator that checks whether the final system state satisfies the task requirement. This may involve inspecting file contents, application states, visible UI text, or other artifacts. The evaluator does not require the agent to follow a single reference trajectory. This is essential because computer tasks often have multiple valid solutions. For example, a document can be printed through a toolbar button, a menu item, a shortcut, or an application command. A benchmark that only recognizes one trajectory would understate agent competence and distort optimization.

OSWorld also groups tasks by category, including operating-system tasks, office tasks, daily applications, professional applications, and multi-application workflows. This taxonomy supports diagnostic analysis. If an agent performs well on OS settings but poorly on Office tasks, the problem may involve GUI manipulation in document-like interfaces. If it performs poorly on workflow tasks, the problem may involve long-horizon planning and cross-application state transfer. Thus, the benchmark is not only a leaderboard but also a diagnostic instrument.

5.2 CogAgent Methodology

CogAgent’s methodology is model-centered. It begins from the observation that GUI screenshots are visually different from natural images and demand higher resolution than many standard VLM pipelines. Small text, icons, and controls are often essential to action selection. A model that compresses the screen too aggressively may preserve overall layout while losing exactly the information needed to click the correct element.

The model builds on CogVLM and introduces a high-resolution cross-module. One visual pathway captures broad layout, while another pathway processes the screenshot at 1120 by 1120 resolution. The high-resolution branch is smaller than the main visual encoder, allowing the model to attend to fine details without exploding computational cost. This design is a compromise between two extremes. Low-resolution-only perception is efficient

but misses key GUI details. Naive high-resolution processing is expensive. CogAgent’s dual-path design attempts to obtain the benefits of high-resolution perception within a manageable compute budget.

Training data is another methodological component. GUI images differ from natural images in distribution, structure, and semantics. The paper constructs GUI- and OCR-related pretraining data so that the model learns interface-specific grounding. This matters because GUI understanding is not merely recognizing text; it is mapping text and visual elements to actionable roles. A text label may name a button, a menu item, a tab, a file, a cell, or an inactive status message. The model must infer which visual entity is relevant to the instruction.

CogAgent is evaluated on both general visual question answering and GUI navigation benchmarks. This dual evaluation is significant. It shows that specialization for GUIs need not destroy general multimodal ability, while also demonstrating gains in the target computer-use setting. In the context of this report, CogAgent supplies a perceptual building block for agentic OS systems: high-resolution, GUI-aware visual-language reasoning.

5.3 OSCAR Methodology

OSCAR’s methodology is system-centered. It treats agentic OS control as a state machine rather than a one-shot prediction problem. The state machine includes initialization, observation, planning, execution, verification, success, failure, reset, and error handling. Transitions are triggered by user requests, planning outputs, execution results, verification feedback, and OS errors. This design makes the agent’s interaction with the environment explicit.

In the observation state, OSCAR captures the current screen and performs GUI grounding. The inspected paper describes a combination of original screenshot input, semantic grounding input, and Set-of-Mark-based visual grounding. In dynamic OS environments, the system can extract Set-of-Mark information using native APIs to obtain accessibility-tree information. In other settings, it can use a detection-plus-OCR pipeline. The purpose is to

convert a raw screen into a representation that supports precise action selection.

In the planning state, OSCAR uses the user request, current observation, context memory, and feedback from prior attempts to decide what to do next. The system uses task-driven re-planning rather than re-planning the entire workflow from scratch at every failure. This is a key distinction. If only one subtask fails, the agent should repair that subtask while preserving useful progress. This reduces redundant attempts and better matches human troubleshooting.

In the execution state, OSCAR uses code-centric control. It generates executable Python code to interact with the OS through mouse, keyboard, and application-control primitives. Code-centric control has two advantages. First, it can be precise and compositional: a script can move to an element, click, type, wait, and check conditions. Second, it provides interpreter feedback when execution fails. Syntax errors, invalid element references, or exceptions can be fed back into the planning state.

In the verification state, OSCAR checks whether the task has been completed. Negative feedback triggers re-planning. This is where dynamic environments differ from static datasets. In a static dataset, the model predicts the next action and is compared to a reference. In OSCAR’s dynamic setting, the agent must reason about whether its own previous action achieved the intended state. This creates a feedback loop between perception, planning, control, and evaluation.

5.4 Methodological Relationship

The three methodologies are complementary. OSWorld defines the environment and evaluation problem. CogAgent improves the perceptual representation needed to interpret GUI screens. OSCAR defines an agent runtime that uses observations and feedback to execute tasks dynamically. A mature agentic OS stack would likely need all three methodological layers. It would need benchmarks and evaluators to measure progress, perception models to read interfaces, and runtime architectures to act safely and adaptively.

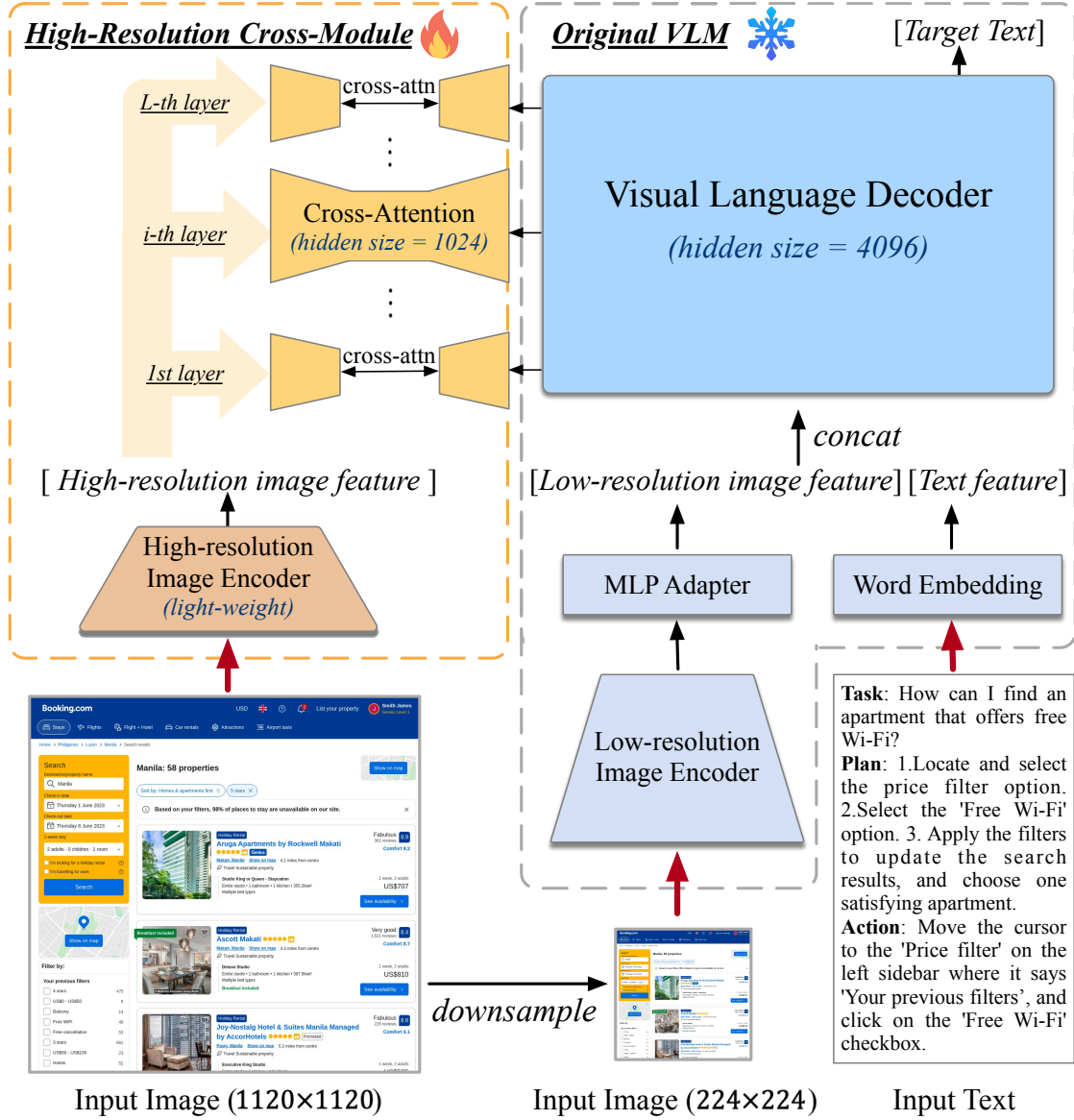


Figure 5.1: CogAgent architecture. The model combines a general visual-language backbone with a high-resolution branch so that global layout and fine-grained GUI details can both contribute to reasoning.



A bride [[592,179,866,956]] and groom [[717,160,862,882]] standing near the water [[425,426,990,898]] at sunset.



Chicken and rice dish [[301,160,997,792]] in a pan [[282,107,998,839]], with spoons [[091,476,630,953]] and plate [[002,004,258,374]] next to it.



A bluebird [[302,413,640,752]] sitting on a branch coffee mugs [[279,588,677,804]].

Figure 5.2: Examples of GUI grounding data used by CogAgent. This figure illustrates why GUI pretraining is different from ordinary image-caption pretraining: the model must connect instructions, text, layout, and actionable visual elements.

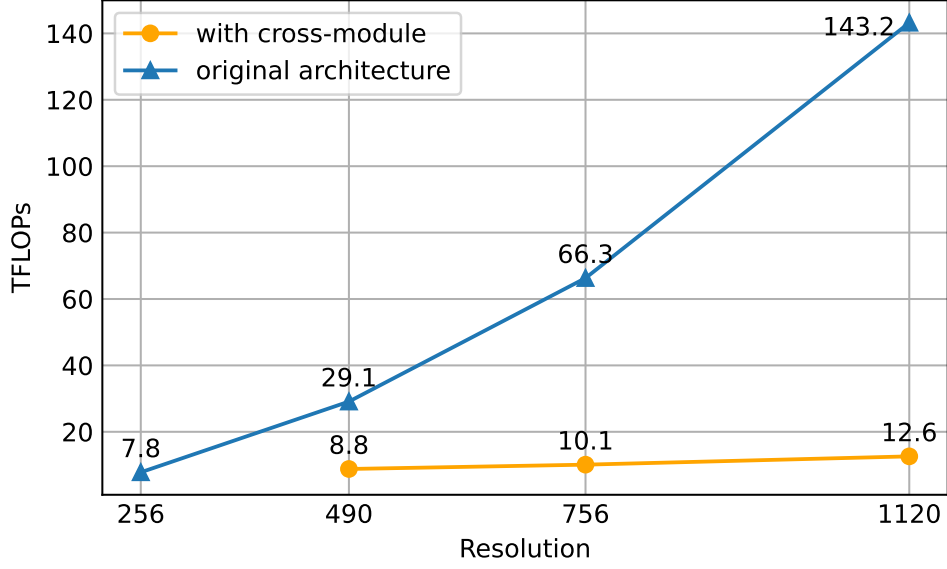


Figure 5.3: CogAgent’s FLOPs comparison illustrates the trade-off between high-resolution GUI perception and computational efficiency. The report uses this figure to explain why GUI perception requires architectural design rather than simple input scaling.

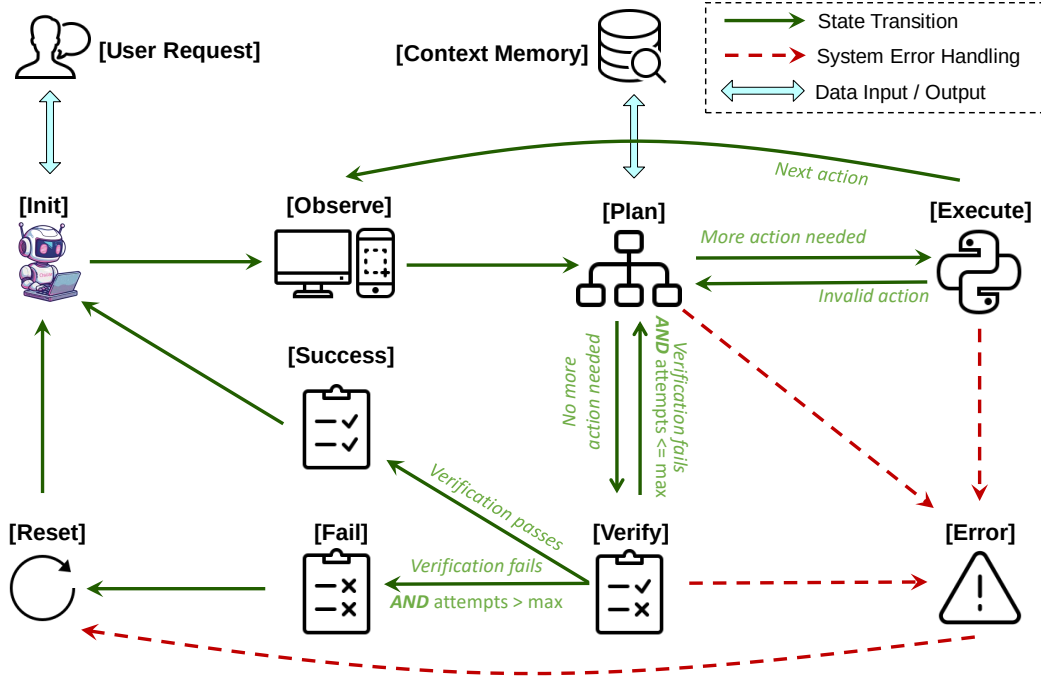


Figure 5.4: OSCAR’s state machine formalizes OS control as repeated observation, planning, execution, verification, and re-planning. This figure is central to the report’s closed-loop control analysis.

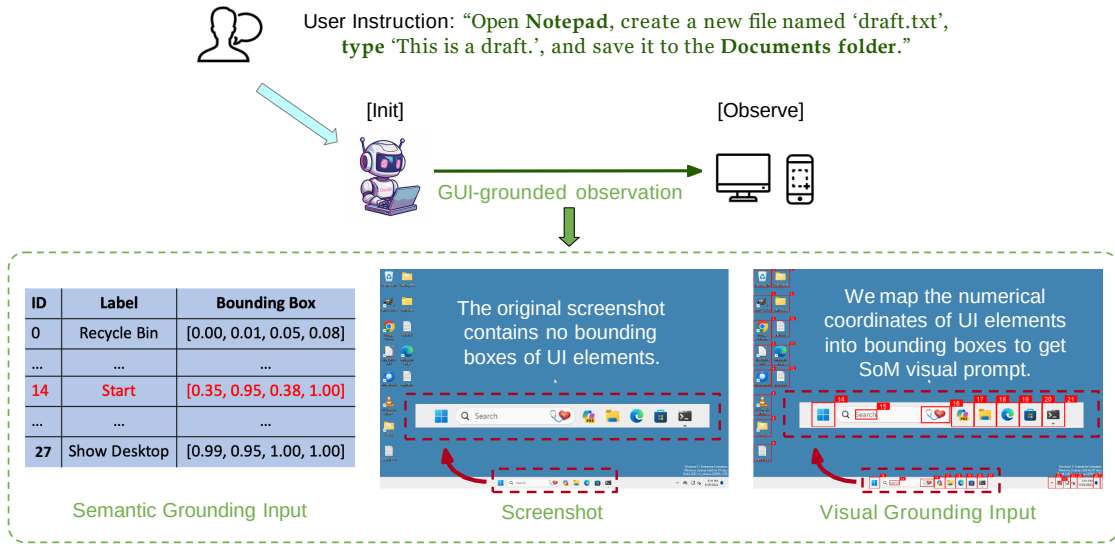


Figure 5.5: OSCAR’s GUI-grounded observation combines raw screenshots with semantic and visual grounding inputs. This multimodal observation design helps the agent identify actionable elements.

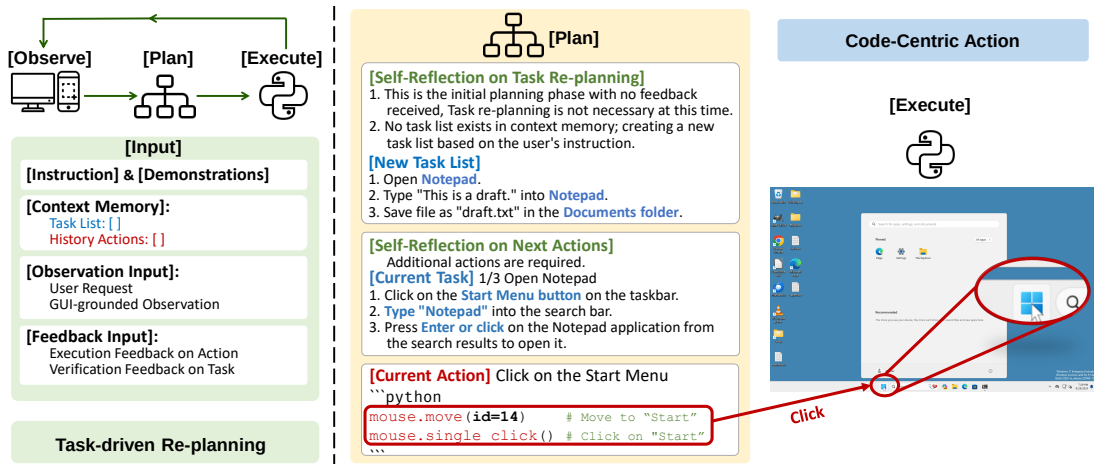


Figure 5.6: OSCAR’s task-driven re-planning refines the current task list and action choice using observation, memory, and feedback. It is designed to avoid restarting the entire workflow after every failure.



Figure 5.7: A qualitative OSCAR example from the source paper. The example shows the kind of multi-step GUI workflow for which state-aware observation, code-centric action, and re-planning are useful.

Chapter 6

Cross-Paper Comparison and Synthesis

6.1 Benchmark, Perception, and Control

The most useful way to synthesize the three papers is to view them as answering three different layers of the same research question. OSWorld asks: how should we measure whether agents can use computers? CogAgent asks: how should an agent perceive dense graphical interfaces? OSCAR asks: how should an agent act and recover in dynamic operating-system environments? These layers are sequential but also mutually reinforcing. A better benchmark reveals perception and control failures; a better perception model improves action selection; a better control loop converts perception into reliable task completion.

This layered interpretation follows the slide deck’s narrative: OSWorld first exposes the real-computer gap, CogAgent targets the GUI grounding gap, and OSCAR closes the execution loop. The transition from one work to the next is not merely chronological. It is conceptual. Static language planning is insufficient because computer use requires grounding. Perception is insufficient because real environments drift and fail. Evaluation is insufficient unless it measures final task success rather than reference-action imitation.

Table 6.1: *Functional positioning of the three core papers in the agentic OS trajectory.*

Dimension	OSWorld	CogAgent	OSCAR
Primary role	Benchmark and real-computer evaluation environment	GUI-specialized visual-language model	Closed-loop OS agent system
Core question	Can agents actually complete real computer tasks?	Can a model perceive and ground dense GUI screenshots?	Can an agent execute, verify, and recover in dynamic OS environments?
Main bottleneck exposed or addressed	Evaluation realism, task setup, execution-based success	High-resolution GUI perception and action grounding	Runtime reliability, feedback, and task-driven re-planning
Key artifact	369 tasks, reproducible environments, execution-based evaluators	18B VLM with low- and high-resolution visual pathways	State-machine agent runtime with code-based action execution
Role in report storyline	Makes the problem measurable	Explains why perception is a necessary layer	Shows why perception must be embedded in control

6.2 Comparison by Environment Realism

OSWorld is strongest on environment realism. Its tasks involve real applications, files, workflows, and execution-based evaluation. It is designed to expose the complexity of actual desktop use. CogAgent’s evaluation includes GUI navigation benchmarks, but its core contribution is not a live OS environment. It is a model designed to understand screenshots. OSCAR adopts dynamic environments such as OSWorld and AndroidWorld to evaluate its control loop. Thus, OSWorld supplies the realism, while OSCAR uses that realism as a testbed for adaptive control.

The distinction matters because environment realism changes what counts as success. In a static benchmark, a model may receive a screenshot and predict the next action. In a dynamic OS environment, the model’s action changes the world, and the next observation may differ from expectation. The agent must handle delays, modal windows, wrong clicks, partial progress, and alternate valid solutions. OSWorld and OSCAR therefore move the

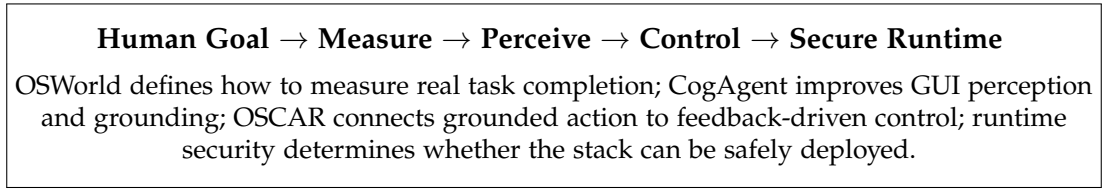


Figure 6.1: A synthesized pipeline for agentic OS control derived from the slide deck and the three core papers. The report’s thesis is that an agentic OS must be measurable, perceptive, controllable, and secure.

field closer to deployment conditions.

6.3 Comparison by Observation and Grounding

CogAgent is strongest on visual observation. It is explicitly designed for high-resolution GUI perception. Its dual-encoder architecture targets the small-text and fine-element problem that standard VLMs struggle with. OSWorld evaluates different observation settings and discovers that grounding remains a major limitation. OSCAR uses GUI-grounded observation as one module in a broader agent system, combining screenshots with semantic and Set-of-Mark inputs.

The papers collectively suggest that no single observation representation is sufficient. Screenshots are universal and faithful to what humans see, but hard to parse precisely. Accessibility trees and structured UI data are precise when available, but incomplete or inconsistent across applications. OCR helps with text, but not with layout or interaction semantics. A robust agentic OS layer should combine these sources and allow them to check one another. For instance, a screenshot can reveal a canvas element missing from the accessibility tree, while the accessibility tree can provide coordinates and labels that the screenshot alone makes hard to infer.

6.4 Comparison by Action Representation

The action representations also differ. OSWorld supports actions through computer-control mechanisms such as mouse and keyboard operations, making it suitable for evaluating

Table 6.2: Comparison of observation, action, feedback, and evaluation assumptions.

Aspect	OSWorld	CogAgent	OSCAR
Observation	Real computer state through screenshots and environment interfaces	Screenshot-centered high-resolution GUI perception	Screenshots, GUI grounding, task state, and execution feedback
Action	Mouse, keyboard, and system-level computer-control actions used by evaluated agents	Benchmark-specific GUI navigation/action prediction	Executable Python control code and GUI actions
Feedback	Environment state changes and final execution-based evaluator results	Mostly static benchmark feedback in navigation datasets	Runtime errors, screenshots, verification signals, and task-driven re-planning
Success criterion	Actual task completion in a reproducible OS environment	Correct GUI understanding or action prediction on benchmark tasks	Successful dynamic task completion across desktop, mobile, and workflow benchmarks
Main risk	Custom evaluators are expensive and coverage remains finite	Strong perception may still fail under dynamic execution	Code-based action increases safety and governance concerns

agents in real software. CogAgent is evaluated primarily as a model that predicts or reasons about GUI navigation actions, so its action representation is tied to benchmark formats. OSCAR uses executable Python code as an action medium, giving it a richer and more compositional control interface.

Code-centric control is powerful but risky. It lets the agent express precise sequences and use OS automation libraries. It also introduces failure modes such as invalid code, unexpected side effects, and security concerns. OSCAR partially addresses this by feeding execution errors back into re-planning, but a deployed agentic OS would need stronger sandboxing, permission control, and audit logs. The comparison therefore shows that action representation is not only a matter of model output format. It is a systems design choice.

6.5 Comparison by Feedback and Re-Planning

Feedback is where OSCAR most clearly extends the other two works. OSWorld evaluates agents in a setting where feedback matters, but it is not itself an agent architecture. CogAgent improves perception but does not by itself define a closed-loop runtime. OSCAR explicitly incorporates feedback through its state machine. Execution errors and verification failures become inputs to subsequent planning.

This is the central step from GUI agents to agentic OS agents. A GUI agent may choose a plausible next click. An agentic OS agent must know whether the click worked, what changed, and what to do if it did not. Task-driven re-planning is especially important because it avoids a common inefficiency: restarting the whole plan after a local failure. In long workflows, full re-planning can cause repetition, contradiction, or loss of useful progress. Fine-grained re-planning preserves context and focuses repair.

6.6 Comparison by Limitations

Each work also has limitations. OSWorld is a benchmark, so it exposes failures more than it solves them. Its custom evaluators are powerful but labor-intensive to create. The benchmark also cannot cover every software environment, credential setting, or user preference. CogAgent improves GUI perception, but perception alone does not guarantee robust task completion in dynamic environments. A model may identify the right button but still fail after a window changes or an action produces an unexpected result. OSCAR improves closed-loop control, but its reported success rates show that reliability remains far from solved. Its code-centric control also raises safety and governance questions.

The synthesis is therefore not that the three papers complete the problem, but that they clarify the problem’s structure. Reliable agentic OS control requires benchmark realism, GUI perception, adaptive runtime design, and security-aware execution. Progress in only one layer is necessary but incomplete.

Table 6.3: *Limitations and open problems revealed by the cross-paper synthesis.*

Open problem	Evidence from the three works	Why it matters	Possible research direction
Benchmark scalability	OSWorld requires task setup, quality control, and custom evaluators	Real software diversity is much larger than any single benchmark	Semi-automatic evaluator generation and benchmark maintenance tools
Grounding uncertainty	OSWorld failure modes and CogAgent ablations show persistent GUI grounding difficulty	Wrong clicks can invalidate long trajectories	Confidence-aware grounding, active perception, and multi-source observation
Execution drift	OSCAR motivates state-machine control because static plans break under dynamic feedback	Small errors accumulate in multi-application workflows	Fine-grained verification, local repair, and trajectory-level memory
Security boundary	OSCAR-style code execution and OS-level agents operate with user authority	Agents can modify files, use credentials, and ingest untrusted content	Sandboxing, permissions, audit logs, and reversible execution
Reproducibility	OS tasks depend on software versions, initial states, and evaluator assumptions	Comparisons can become unstable over time	Containerized environments, versioned tasks, and evaluator transparency

Chapter 7

Experimental Results and Analysis

7.1 Why More Tables Are Needed

The three source papers are empirical papers, so the experimental section should not only summarize their conclusions in prose. It should also preserve the main quantitative evidence that supports the report’s synthesis. This chapter therefore includes representative tables from OSWorld [22], CogAgent [5], and OSCAR [20]. The tables are reformatted to match the report template, but the reported values follow the source papers. I focus on tables that clarify the central storyline: OSWorld measures the real-computer gap; CogAgent demonstrates the value of high-resolution GUI perception and GUI-specific data; and OSCAR demonstrates the value of state-aware re-planning in dynamic environments.

7.2 OSWorld: Baseline Gap on Real Computer Tasks

OSWorld evaluates a wide set of LLM and VLM agents, including GPT-family models [10,11], Gemini-family models [4], Claude-3 Opus [1], Qwen-Max [2], Mixtral [6], Llama-3 [8], and CogAgent [5]. The benchmark also studies different observation settings, including accessibility trees, raw screenshots, screenshot plus accessibility tree, and Set-of-Mark prompting [23]. Table 7.1 reproduces the main baseline pattern from the OSWorld paper.

The most important lesson is the gap between models and humans. Even when GPT-4

Table 7.1: Representative OSWorld baseline results. Success rates are grouped by task category. The table highlights the large gap between current LLM/VLM agents and human users in real-computer environments.

Input	Model	OS	Office	Daily	Prof.	Workflow	Overall
A11y tree	Mixtral-8x7B [6]	12.50	1.01	4.79	6.12	0.09	2.98
	Llama-3-70B [8]	4.17	1.87	2.71	0.00	0.93	1.61
	GPT-3.5 [10]	4.17	4.43	2.71	0.00	1.62	2.69
	GPT-4 [11]	20.83	3.58	25.64	26.53	2.97	12.24
	Gemini-Pro [4]	4.17	1.71	3.99	4.08	0.63	2.37
	Qwen-Max [2]	29.17	3.58	8.36	10.20	2.61	6.87
	GPT-4o	20.83	6.99	16.81	16.33	7.56	11.36
Screenshot	CogAgent [5]	4.17	0.85	2.71	0.00	0.00	1.11
	GPT-4V [11]	12.50	1.86	7.58	4.08	6.04	5.26
	Gemini-ProV [4]	8.33	3.58	6.55	16.33	2.08	5.80
	Claude-3-Opus [1]	4.17	1.87	2.71	2.04	2.61	2.42
Screenshot + A11y	GPT-4V [11]	16.66	6.99	24.50	18.37	4.64	12.17
	Claude-3-Opus [1]	12.50	3.57	5.27	8.16	1.00	4.41
	GPT-4o	41.67	6.16	12.33	14.29	7.46	11.21
Set-of-Mark	GPT-4V [11,23]	8.33	8.55	22.84	14.28	6.57	11.77
	Gemini-Pro-1.5 [4]	16.67	5.13	12.96	10.20	3.60	7.79
	Claude-3-Opus [1]	12.50	2.72	14.24	6.12	4.49	6.72
Human Performance		75.00	71.79	70.51	73.47	73.27	72.36

with accessibility-tree input reaches 12.24% overall success, human users reach 72.36%. The gap is not uniform across task types. Models are relatively stronger on some OS and daily-application tasks, but workflow tasks and GUI-heavy office tasks remain difficult. This supports the OSWorld claim that real computer use requires more than text reasoning or isolated visual recognition. It requires precise GUI grounding, operational knowledge, and robust execution over long trajectories.

7.3 CogAgent: GUI Perception and Navigation Evidence

CogAgent is evaluated on GUI navigation benchmarks such as Mind2Web [3] and Android in the Wild (AITW) [13]. These benchmarks are not identical to OSWorld: they are closer to action prediction or navigation tasks. However, they are useful because they isolate the perception and grounding abilities that later become necessary in dynamic OS agents.

The Mind2Web and AITW results support the perception part of the report’s argument.

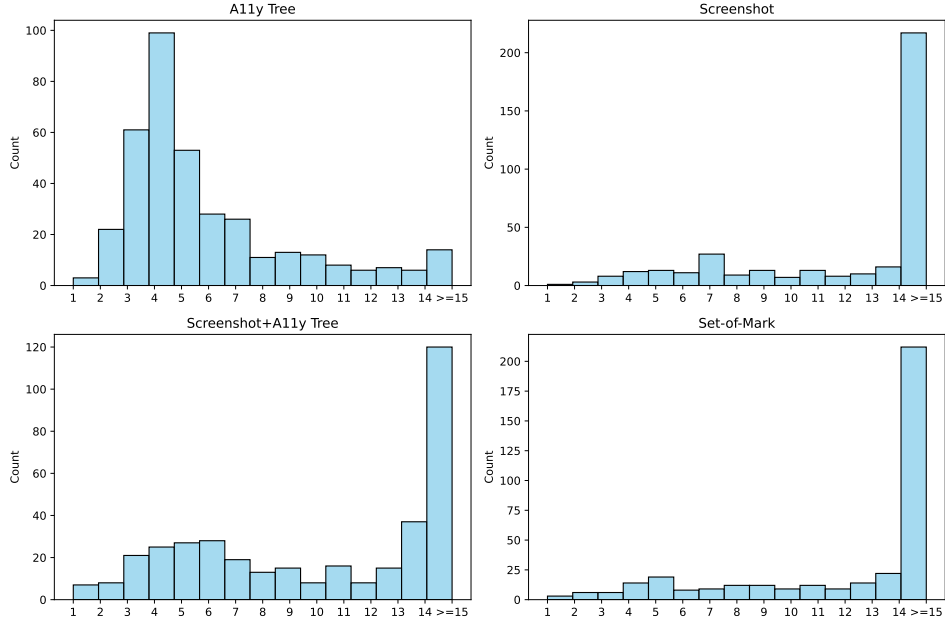


Figure 7.1: OSWorld analysis of agent step counts. The figure helps explain why long-horizon tasks are difficult: more steps create more opportunities for grounding errors, wrong operations, and state drift.

Table 7.2: CogAgent results on Mind2Web. The comparison shows that screenshot-based GUI perception can outperform HTML-based LLM methods in this setting.

Method	Cross-task	Cross-website	Cross-domain	Overall
<i>Screen representation: HTML</i>				
GPT-3.5 [10]	18.6	17.4	16.2	17.4
GPT-4 [11]	36.2	30.1	26.4	30.9
Flan-T5 XL [3]	52.0	38.9	39.6	43.5
LLaMA2-7B [17]	52.7	47.1	50.3	50.1
LLaMA2-70B [17]	55.8	51.6	55.7	54.4
<i>Screen representation: image</i>				
Qwen-VL [2]	12.6	10.1	8.0	10.2
CogVLM [19]	37.1	23.4	26.3	23.9
CogAgent [5]	62.3	54.0	59.4	58.2

CogAgent is not simply a larger generic VLM; it is adapted to GUI understanding and navigation. Its performance suggests that screenshot-based agents can become competitive when the visual model has sufficient resolution and GUI-specific grounding.

Table 7.3: CogAgent results on Android in the Wild. The table shows that CogAgent improves over textual-description baselines and Auto-UI on the overall AITW score.

Method	GoogleApp	Install	WebShop	General	Single	Overall
<i>Screen representation: textual description (OCR + icon)</i>						
GPT-3.5 [10]	10.47	4.38	8.42	5.93	9.39	7.72
LLaMA2-7B [17]	30.99	35.18	19.92	28.56	27.35	28.40
<i>Screen representation: image</i>						
Auto-UI [25]	71.37	76.89	70.26	68.24	84.58	74.27
CogAgent [5]	74.95	78.86	71.73	65.38	93.49	76.88

Table 7.4: CogAgent ablation on pre-training data. Adding GUI and grounding data produces a large Mind2Web improvement, supporting the claim that GUI-specific data matters.

Pre-train data	Base res.	Cross res.	STVQA	DocVQA	Mind2Web
Caption	490	–	68.1	57.6	38.6
Caption + OCR	490	–	72.5	59.8	40.7
Caption + OCR	224	1120	78.2	74.1	41.4
All GUI/Grounding Data	224	1120	79.4	76.4	54.2

Table 7.4 is especially important for synthesis. OCR improves text-rich perception, but the largest Mind2Web gain comes when the model receives broader GUI and grounding data. This supports the idea that GUI grounding is a specialized capability. It is related to OCR, but not reducible to OCR.

7.4 OSCAR: Dynamic Control Results

OSCAR evaluates closed-loop control on GAIA [9], OSWorld [22], and AndroidWorld [14]. The comparison baselines include UFO [26], FRIDAY [21], MMAC [15], Cradle [16], AppAgent [24], M3A [14], and Mobile Agent [18]. These citations matter because OSCAR positions itself against agents that already attempt real or semi-real workflow automation, not only against static GUI-navigation models.

The OSCAR tables show a consistent pattern. State-aware, task-driven re-planning improves average performance, especially as tasks become longer and more dynamic. However, the results also reveal that the problem is far from solved. A 24.5% OSWorld

Table 7.5: OSCAR results on GAIA. The table reports exact-match scores across increasing workflow complexity.

Model	Level 1	Level 2	Level 3	Average
GPT-4-turbo [11]	9.7	6.9	0.0	5.5
GPT-4 plugins [11]	30.3	9.7	0.0	13.3
UFO [26]	36.9	16.1	5.4	19.4
FRIDAY [21]	40.9	20.1	6.1	22.4
MMAC [15]	45.2	20.8	6.1	24.0
OSCAR [20]	47.0	25.6	13.5	28.7

Table 7.6: OSCAR results on OSWorld. All baselines use SoM visual prompts and GPT-4o in the OSCAR paper’s comparison.

Model	OS	Office	Daily	Prof.	Multi	Avg.
Cradle [16]	16.7	3.5	6.6	20.4	5.5	10.5
UFO [26]	37.5	6.8	12.8	14.3	10.9	16.5
FRIDAY [21]	45.8	8.5	14.1	18.4	6.9	18.8
OSCAR [20]	58.3	12.0	16.7	22.4	12.9	24.5

Table 7.7: OSCAR results on AndroidWorld. The table shows that OSCAR improves average performance over AppAgent, Mobile Agent, and M3A in the reported setting.

Model	Easy	Medium	Hard	Average
M3A [14]	41.0	33.3	26.3	33.5
Mobile Agent [18]	49.2	41.7	31.6	40.8
AppAgent [24]	82.0	55.6	42.1	59.9
OSCAR [20]	65.6	66.7	52.6	61.6

success rate improves substantially over baselines, but it remains far below human-level performance reported by OSWorld. Therefore, OSCAR should be interpreted as evidence for the value of closed-loop runtime design, not as evidence that general OS automation is complete.

7.5 Cross-Paper Experimental Lessons

Several experimental lessons emerge across the papers.

First, current agents are far below reliable human computer use. OSWorld’s human-agent gap remains the strongest evidence. Even OSCAR’s improved OSWorld result of 24.5% is still far from human-level reliability. The practical implication is that current systems may be useful for assisted automation or constrained workflows, but should not be treated as universally reliable autonomous operators.

Second, GUI grounding is a persistent bottleneck. OSWorld identifies it as a failure mode, CogAgent directly improves it through high-resolution visual modeling and GUI-specific data, and OSCAR includes GUI-grounded observation as an essential component. The convergence across all three works makes grounding one of the most supported claims in the report.

Third, dynamic feedback changes the problem. Static benchmarks such as Mind2Web [3] and AITW [13] can measure action prediction, but they cannot fully measure recovery, alternate paths, or final-state success. OSWorld and OSCAR show that real computer tasks require execution-based evaluation and re-planning.

Fourth, operational knowledge matters. Agents may have broad language knowledge but still fail in software-specific procedures. Professional tools are especially challenging because they require knowledge of menus, shortcuts, modes, file formats, and task-specific conventions. Future agents may need retrieval, tutorials, demonstrations, or application manuals integrated into the runtime.

Fifth, long-horizon reliability remains the core obstacle. Errors accumulate over time. A single failure in a ten-step workflow may be recoverable; multiple failures in a multi-application workflow may derail the task entirely. OSCAR’s task-driven re-planning reduces redundant repair, but the results show that long workflows remain difficult.

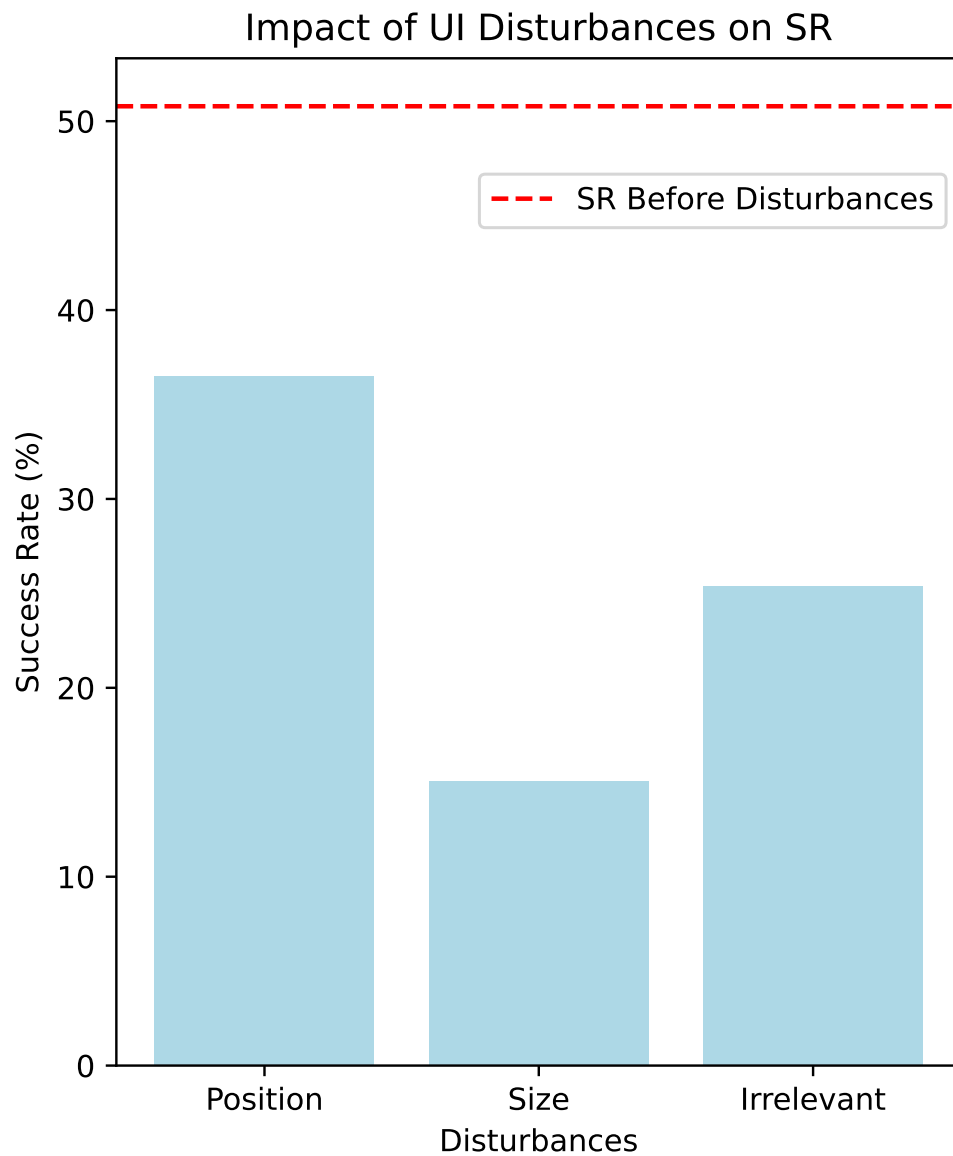


Figure 7.2: OSWorld robustness analysis under UI perturbations. The figure supports the report’s claim that real OS agents must handle noise, unexpected windows, and visual-state variation rather than assuming a clean static screen.

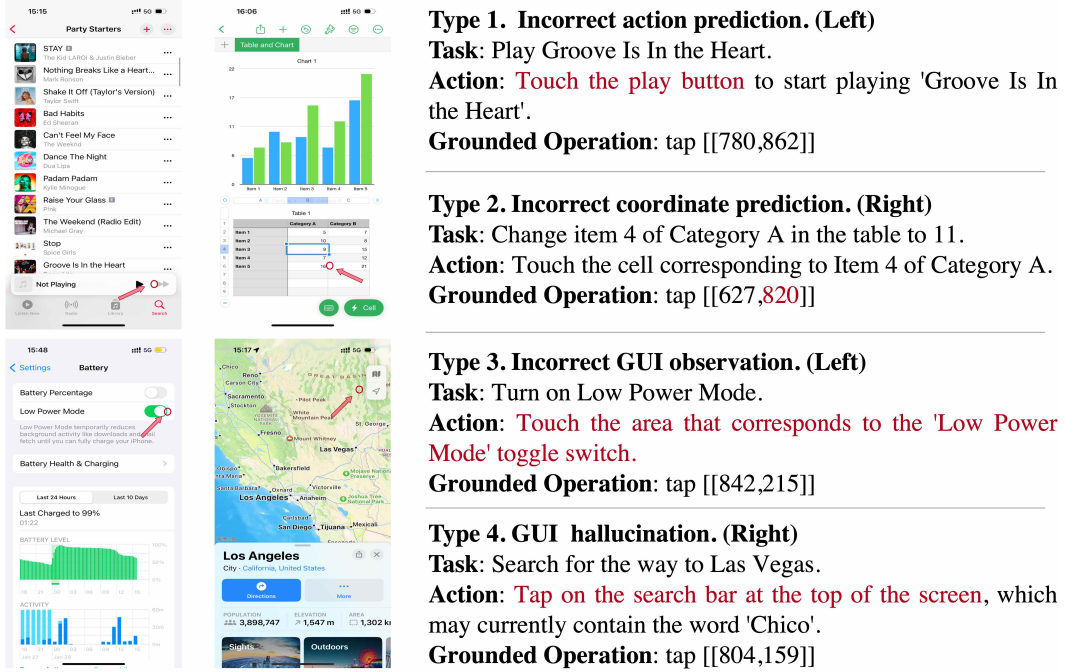


Figure 7.3: CogAgent failure cases from the source material. These examples show that even a GUI-specialized VLM can still struggle with precise interaction, ambiguity, or visual reasoning under complex interface states.

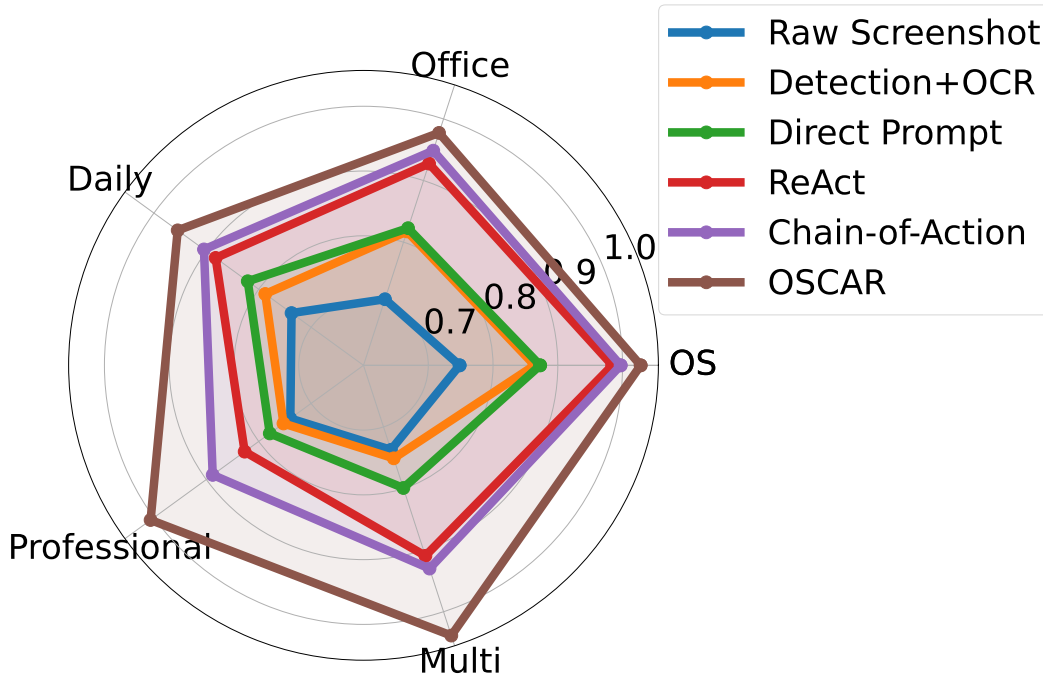


Figure 7.4: OSCAR ablation decomposition. The figure visualizes the contribution of GUI-grounded observation and task-driven re-planning, supporting the claim that both perception and control-loop design are necessary.

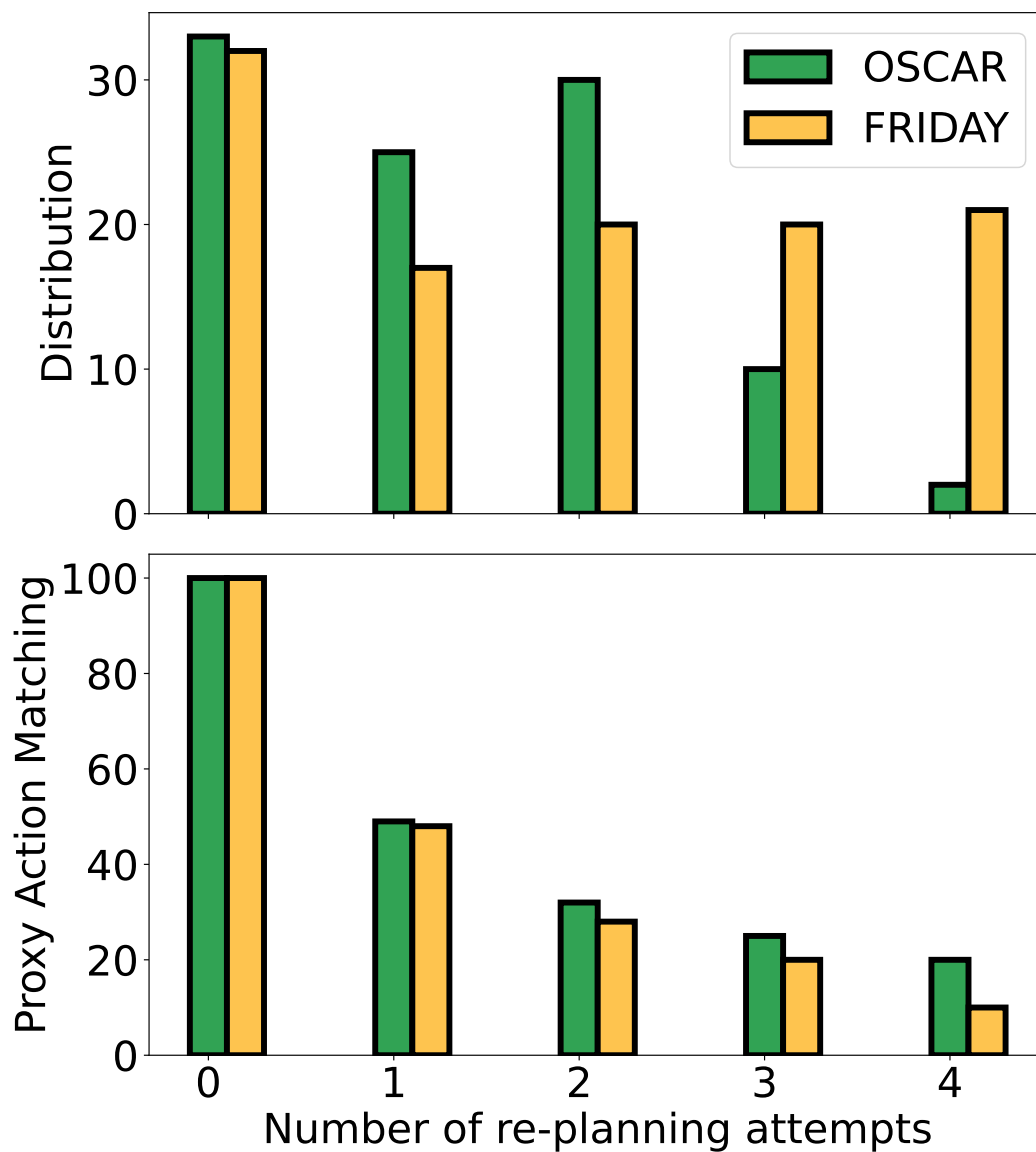


Figure 7.5: OSCAR planning-efficiency analysis for successful cases. The figure illustrates why fine-grained re-planning can be more efficient than repeatedly restarting an entire workflow.

Chapter 8

Discussion and Future Directions

8.1 Strengths of the Three-Work Trajectory

The main strength of the OSWorld–CogAgent–OSCAR trajectory is that it decomposes a broad and ambiguous goal into researchable layers. “Make an AI use a computer” is too vague to guide progress. OSWorld turns it into an evaluation problem, CogAgent turns it into a perception problem, and OSCAR turns it into a control problem. This decomposition is productive because each layer has measurable artifacts: benchmark tasks and success rates, GUI grounding and navigation accuracy, and dynamic task completion under feedback.

Another strength is that the trajectory respects the nature of real computer use. Many earlier agent demonstrations focus on narrow web tasks, API calls, or scripted environments. The inspected works instead confront desktop and mobile applications, multi-application workflows, files, visual interfaces, and dynamic states. This makes the research closer to the environment where users actually want help.

A third strength is the movement from static prediction to closed-loop operation. Static action prediction is useful, but it does not capture the lived reality of software use. OSCAR’s state machine makes failure and repair part of the agent design. This is a more realistic foundation for OS-level automation because even human users routinely adjust after observing feedback.

Task Instruction: I downloaded an episode of Friends to practice listening, but I don't know how to remove the subtitles. Please help me remove the subtitles from the video and export it as "subtitles.srt" and store it in the same directory as the video.

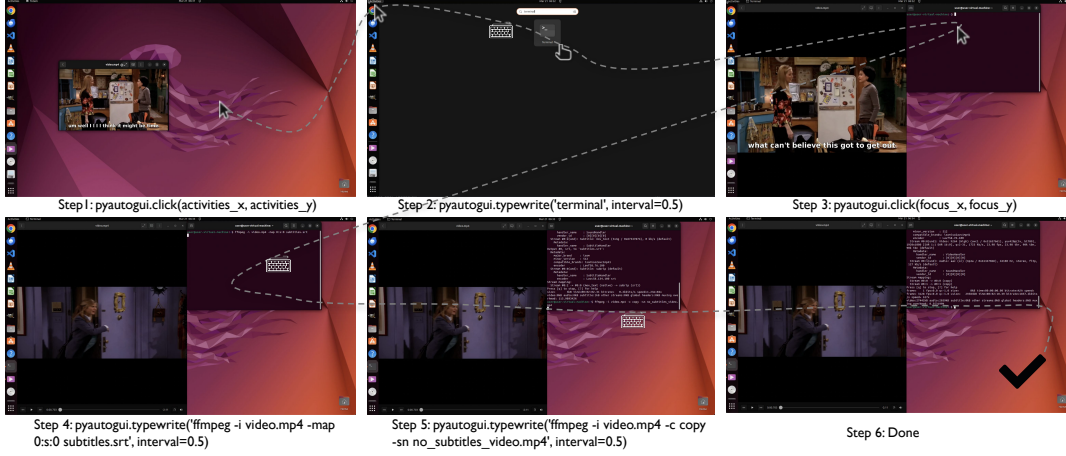


Figure 8.1: A successful OSWorld qualitative example. The case demonstrates that current agents can sometimes solve complex real-computer tasks when perception, planning, and operational knowledge align.

8.2 Weaknesses and Remaining Limitations

Despite these strengths, the trajectory also reveals limitations. OSWorld’s benchmark construction is expensive because realistic tasks require setup scripts and custom evaluators. This raises scalability questions. If every new application domain requires extensive human effort to define evaluators, benchmark coverage will lag behind the diversity of real software.

CogAgent’s limitation is that perception is only one layer. High-resolution GUI understanding can improve target selection, but it does not guarantee task success. A model may still lack application-specific knowledge, misunderstand a user’s goal, or fail when the environment changes after an action. Perception models also face practical cost constraints when used repeatedly in long-running agents.

OSCAR’s limitation is that closed-loop control improves robustness but does not solve reliability. A 24.5% success rate on OSWorld is an improvement, yet it is still far below what would be required for autonomous deployment in high-stakes settings. Code-centric control also expands the risk surface. When an agent can run code and interact with files, safety policies and runtime constraints become essential.

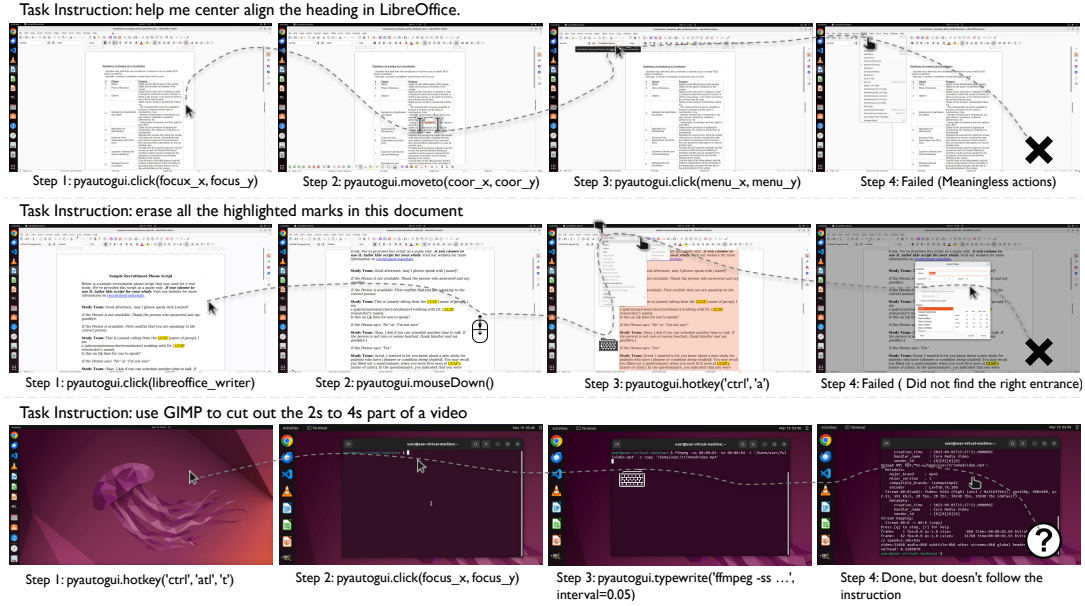


Figure 8.2: OSWorld qualitative failure examples. These cases illustrate grounding failure, missing operational knowledge, and action-state mismatch, which motivate the need for both GUI-specialized perception and closed-loop recovery.

8.3 Validity of the Cross-Paper Comparison

The three papers should not be compared as if they were competing systems on a single leaderboard. OSWorld is primarily a benchmark contribution, CogAgent is primarily a perception-model contribution, and OSCAR is primarily a runtime-agent contribution. Their evaluation settings, metrics, and action spaces differ. Therefore, a higher number in one table does not automatically imply a globally better agentic OS design. The value of the comparison is structural: it shows which capability each paper isolates and why the capabilities must eventually be combined.

The report also depends on the source papers’ own experimental protocols. It does not independently reproduce OSWorld tasks, retrain CogAgent, or rerun OSCAR. This creates three validity constraints. First, benchmark results may depend on software versions, environment setup, model access, and prompt details. Second, GUI-agent performance can be sensitive to visual resolution, accessibility metadata, and implementation choices that are not always identical across systems. Third, closed-loop agent results can vary with retries,

time limits, tool availability, and failure-handling policy. These constraints do not invalidate the papers, but they require caution when turning reported success rates into broad claims about deployment readiness.

8.4 Benchmark and Evaluation Risks

Execution-based benchmarks are a major improvement over static action matching, but they introduce their own risks. Custom evaluators may encode narrow definitions of task success. A task may have several valid final states, while an evaluator recognizes only some of them. Conversely, a benchmark may check a final file or UI state without capturing whether the agent took unsafe intermediate actions. Future OS benchmarks should therefore report not only success rate but also trajectory quality, number of corrections, unsafe actions, time cost, and user interventions.

Another evaluation risk is benchmark saturation through task-specific adaptation. If agents are repeatedly tuned to a fixed set of tasks, success may reflect benchmark familiarity rather than general computer-use ability. OSWorld’s emphasis on realistic task diversity helps, but long-term progress will require continually refreshed tasks, hidden test environments, and evaluator transparency.

8.5 Perception and Action Risks

CogAgent demonstrates that high-resolution GUI perception matters, but perception remains uncertain in exactly the places where operating-system actions are consequential. A small icon, a disabled button, a selected cell, or a modal dialog can change the meaning of the next action. The agentic OS layer should therefore treat grounding as a probabilistic decision rather than a guaranteed lookup. When uncertainty is high, the agent should zoom, inspect, query another representation, or ask the user.

Action representation creates a second risk. Mouse and keyboard actions are faithful to human interaction but brittle and slow. Code-based actions are more precise and composi-

tional but can bypass interface affordances and create larger side effects. Accessibility-tree actions offer semantic structure when available but are not uniformly reliable across applications. A mature agentic OS will likely need a hybrid action layer that chooses the least risky action representation for the current task.

8.6 Security and Runtime Boundaries

The slide deck ends by noting that runtime becomes the new security boundary. This is a crucial point. Traditional operating systems already enforce boundaries among users, processes, files, networks, and devices. An agentic layer introduces a new actor that operates on behalf of the user. It may read screens, use credentials, access files, remember context, execute tools, and mutate system state. The agent therefore becomes a bearer of authority.

This creates several risks. Prompt injection can occur through documents, web pages, emails, or application content that the agent reads. Credential misuse can occur if the agent is allowed to act across accounts without clear permission. File damage can occur if the agent misunderstands a task or executes unsafe code. Privacy leakage can occur if observations or memory contain sensitive information. Persistence can create long-term risk if the agent accumulates state that is later misused.

A future agentic OS should therefore include explicit permission models, sandboxed execution, audit logs, reversible actions, user confirmation for high-impact operations, and isolation between trusted user instructions and untrusted environmental content. These mechanisms are not optional add-ons. They are part of the operating-system nature of the problem.

8.7 Case Study: OpenClaw and ClawJacked

The runtime-security concern is not hypothetical. The slide deck uses the OpenClaw “ClawJacked” incident as a concrete example of how agent runtimes can become attack surfaces. Public reports describe ClawJacked as a high-severity vulnerability in OpenClaw’s

local gateway: a malicious website could initiate a WebSocket connection to a locally running OpenClaw agent, exploit weak localhost trust assumptions, and potentially take control of the agent runtime [7, 12]. The important lesson is not limited to one product. It is that browser content, local services, credentials, and autonomous agent execution can intersect in unexpected ways.

This case illustrates why an agentic OS layer must be designed as a security-sensitive systems component rather than a simple model wrapper. A local agent gateway may appear safe because it binds to localhost, but web browsers can still interact with local network endpoints under certain conditions. A password or pairing flow may appear sufficient, but it may fail if local connections are implicitly trusted, if rate limits are relaxed, or if device registration is automatic. Once the attacker controls the agent runtime, the agent’s permissions become the attacker’s permissions: file access, tool calls, logs, connected services, and possibly shell execution.

ClawJacked therefore strengthens the report’s fourth requirement: after measure, perceive, and control, an agentic OS must also secure. Security should be part of the runtime architecture from the beginning. Concrete design requirements include strict origin checks for local gateways, loopback rate limiting, explicit pairing prompts, least-privilege credentials, capability-scoped tools, sandboxed code execution, visible audit trails, and kill-switch controls. Without these mechanisms, improvements in OSWorld-style task success or OSCAR-style control may actually increase risk, because the agent becomes more capable while still operating inside a weak trust boundary.

8.8 Future Direction: Better Grounding

The first future direction is better grounding. GUI grounding should combine visual, textual, structural, and temporal signals. A future agent should know not only where a button appears in the current screenshot, but also how that element relates to the application state, whether it is enabled, what action it triggers, and whether it has changed since the last observation. This suggests models that jointly reason over screenshots, accessibility trees,

OCR, UI histories, and application-specific schemas.

Grounding should also become uncertainty-aware. If the agent is unsure which element corresponds to an instruction, it should ask for clarification, zoom in, use OCR, query the accessibility tree, or move cautiously rather than guessing. Robust agents may need explicit confidence estimation and active perception actions.

8.9 Future Direction: Memory and Operational Knowledge

The second future direction is operational knowledge. Many failures arise not because the agent cannot see the screen, but because it does not know how to use the software. Future systems could integrate manuals, tutorials, help pages, demonstrations, and learned procedural memories. An agent using GIMP, LibreOffice, VS Code, or a browser should be able to retrieve relevant operational patterns and adapt them to the current UI.

Memory must be designed carefully. Short-term memory helps track task progress, previous actions, and failed attempts. Long-term memory may store user preferences and reusable workflows. However, memory also creates privacy and security concerns. The agentic OS layer must decide what to remember, for how long, under whose authority, and with what visibility to the user.

8.10 Future Direction: Verification and Recovery

The third future direction is stronger verification. Current agents often rely on visual inspection or benchmark-specific evaluators. A deployed agent needs broader verification strategies. It should inspect files after writing them, compare visible states with expected outcomes, use application APIs when safe, and request user confirmation when task success is ambiguous. Verification should be treated as a first-class component, not merely a final stop condition.

Recovery should also become more structured. Instead of repeatedly asking a model to try again, the runtime can maintain a failure taxonomy: wrong element, unavailable file,

blocked dialog, invalid code, missing permission, exceeded time, or ambiguous instruction. Different failures require different repair strategies. OSCAR's task-driven re-planning is a step in this direction, but future systems can make recovery more systematic.

8.11 Future Direction: Human-Agent Collaboration

The fourth future direction is collaboration. The most realistic near-term agentic OS is not a fully autonomous replacement for the user, but a cooperative layer. The agent can propose plans, execute low-risk steps, ask before high-risk actions, and leave a clear trace of what it did. This turns automation into a dialogue rather than a black box.

Collaboration also helps with evaluation. A human-in-the-loop agent may succeed at tasks that a fully autonomous agent would fail, because the user can resolve ambiguity, provide credentials, or approve irreversible steps. Future benchmarks could measure not only autonomous success rate but also assisted success rate, number of user interventions, time saved, and error recovery quality.

Chapter 9

Conclusion

This report has synthesized OSWorld, CogAgent, and OSCAR as a connected research path toward agentic operating-system control. The central claim is that LLM-based computer-use agents require more than language reasoning. Following the slide deck’s storyline, the required stack can be summarized as **measure** → **perceive** → **control** → **secure**. Agents must be measured in realistic environments, perceive dense GUIs accurately, control dynamic systems through feedback, and operate inside secure runtime boundaries.

OSWorld establishes the benchmark foundation. By evaluating agents in real computer environments with setup configurations and execution-based evaluators, it shows that existing agents remain far below human performance. Its results reveal practical bottlenecks in GUI grounding, operational knowledge, noise handling, and long-horizon workflow execution.

CogAgent addresses the perception foundation. By specializing a visual language model for GUI understanding and navigation, and by combining global layout perception with high-resolution detail perception, it shows that GUI agents need models adapted to the visual and semantic structure of interfaces. Its ablation evidence supports the importance of GUI and grounding data beyond generic captioning or OCR.

OSCAR addresses the control foundation. By modeling OS interaction as a state machine with observation, planning, execution, verification, and task-driven re-planning, it moves

from static action prediction toward adaptive operation in dynamic environments. Its results on GAIA, OSWorld, and AndroidWorld suggest that closed-loop control improves robustness, although substantial reliability gaps remain.

Together, the three works point toward an agentic OS layer as a new abstraction for human-computer interaction. Such a layer would translate natural-language goals into situated actions across applications, but it would also need strong safety boundaries, permission models, runtime isolation, memory governance, and transparent verification. The OpenClaw/ClawJacked case discussed in the report makes this final point concrete: once agents run with real identity and execution authority, the runtime itself becomes part of the security boundary. The path from current GUI agents to reliable agentic operating systems is therefore both a machine-learning problem and a systems problem. The next stage of research should treat evaluation, perception, control, and security as one integrated stack.

References

- [1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. Technical report, 2024.
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [3] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023.
- [4] Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [5] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [6] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [7] Ravie Lakshmanan. Clawjacked flaw lets malicious sites hijack local open-claw ai agents via websocket. <https://thehackernews.com/2026/02/clawjacked-flaw-lets-malicious-sites.html>, Feb 2026. Accessed: 2026-05-02.
- [8] Meta AI. The llama 3 herd of models. Technical report, 2024.
- [9] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- [10] OpenAI. Chatgpt. <https://openai.com/blog/chatgpt>, 2022.
- [11] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [12] OpenClaw. Clawjacked: How a website could hijack your open-claw agent (and the fix). <https://openclawai.io/blog/clawjacked-vulnerability-what-happened/>, Mar 2026. Accessed: 2026-05-02.

- [13] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. *arXiv preprint arXiv:2307.10088*, 2023.
- [14] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- [15] Yuxuan Song, Chi Zhang, et al. Mmac-copilot: Multi-modal agent collaboration operating system copilot. *arXiv preprint arXiv:2404.18074*, 2024.
- [16] Weihao Tan, Wentao Zhang, Yuxuan Guo, Hao Li, et al. Towards general computer control: A multimodal agent for red dead redemption ii as a case study. *arXiv preprint arXiv:2403.03186*, 2024.
- [17] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [18] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024.
- [19] Weihang Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, et al. Cogvlm: Visual expert for pretrained language models. *arXiv preprint arXiv:2311.03079*, 2023.
- [20] Xiaoqiang Wang and Bang Liu. Oscar: Operating system control via state-aware reasoning and re-planning. *arXiv preprint arXiv:2410.18963*, 2024.
- [21] Zhiyong Wu, Zhen Wang, Xu Xu, Jiwen Lu, Ming Yan, et al. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024.
- [22] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Jing Hua Toh, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.
- [23] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv preprint arXiv:2310.11441*, 2023.
- [24] Zhen Yang, Ji Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*, 2023.

- [25] Zhuosheng Zhan, Aston Zhang, Enze Xie, Yuan Yao, Qian Sun, and Zhe Wu. You only look at screens: Multimodal chain-of-action agents. *arXiv preprint arXiv:2309.11436*, 2023.
- [26] Chaoyun Zhang, Liquun Li, Shilin He, Xu Zhang, Yu Qiao, Lidong Qin, Si Li, et al. Ufo: A ui-focused agent for windows os interaction. *arXiv preprint arXiv:2402.07939*, 2024.